

chapter- one

'Introduction to Data structures'

Data structure are the specific way of organizing data in a specialized format on a computer so that the information can be organized, processed, stored and retrieved quickly and effectively.

Data structure may be organized in many different ways.

The logical or mathematical model of a particular organization of data is called data structures.

It is particular way of organizing data in a computer so that it can be used effectively.

Data structure can be classified as:-

1. Linear data structure:-

→ It arrange data in a linear sequence, such as found in array, list or queue.

Types: i. Array

ii. Stack

iii. Linked list

iv. Queue.

2. Non-linear data structure:-

→ is a data structure in which insertion and deletion is not possible in a linear fashion.

Types: i. Graph

ii. Tree.

Abstract Data Type (ADT):-

→ A useful tool for specifying the logical properties of a data type is ADT. It is a way of classifying data structures based on how they are used and the behaviours they provide. They do not specify how the data structure must be implemented or laid out in a memory, but simply provide a minimal expected interface and set of behaviours.

It refers to the basic mathematical concept, that defines the data type.

In general terms, an ADT is an specification of the values and operations that has two properties:-

- i. It specifies everything you need to know in order to use the data type.
- ii. It makes absolutely no reference to the manner in which the data type will be implemented.

The data structure is basically a technique of organizing and storing of different types of data items in computer memory. A data structure is the portion of memory allotted for a model, in which the required data can be arrange in a proper fashion. Data structure mainly specifies the following four things:-

- i. organization of data
- ii. Accessing methods.
- iii. Degree of associativity
- iv. Processing alternative for information.

Linear Data structure

i. They are used to represent sequential data.

ii. It is easy to implement.

iii. The basic linear data structures are list, stack and queue.

iv. Data element can be ~~traversed~~ traversed in a single run only.

v. It is used in application software application.

Non linear structure

i. They are used to represent hierarchical data.

ii. It is used to implement.

iii. The basic non-linear data structures are trees and graphs.

iv. Data elements cannot be traversed in a single run only.

v. It is used in AI and image processing.

• Why is Data structure important:- (Advantage).

i. It helps to store information securely in computer system.

ii. It helps to access and process data efficiently.

iii. By help of data structure, we can traverse all elements/data easily and efficiently.

(Disadvantage).

i. Advanced IT/computer expertise is required to alter the data in data structure.

ii. It is hard to implement and manage efficiently.

AVL TreeTree Structure

AVL Tree:-

→ self balancing binary search tree where the height of left and right subtree for any node cannot be more than one.

The differences between the height of left subtree and right subtree is called balancing factor.

$B.F = \text{Height of Left Subtree} - \text{Height of Right Subtree}$

To become balance, the BF must $\in [0, 1, -1]$.

Balancing Algorithm:-

- i. Left Algorithm.
- ii. Right Algorithm.
- iii. Left-Right Algorithm.
- iv. Right-Left Algorithm.

- Construct AVL Tree from following:
35, 50, 40, 25, 30, 60, 78, 20, 28.

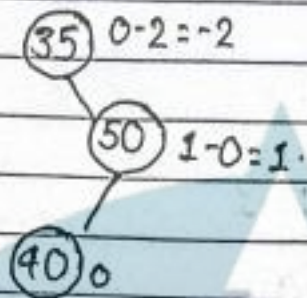
Step 1:- Insert 35

(35)

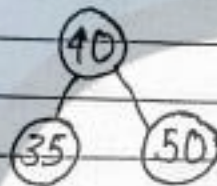
Step 2:- Insert 50

(35) 0-1:-1
 (50) 0

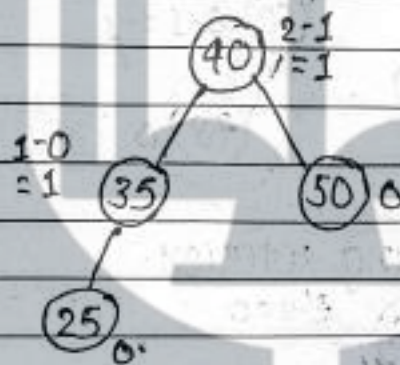
step 3:- Insert 40



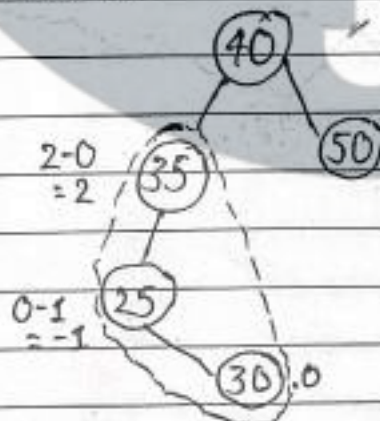
step 4: Performing Rotation (Note:- Median hai top hai lekhe)



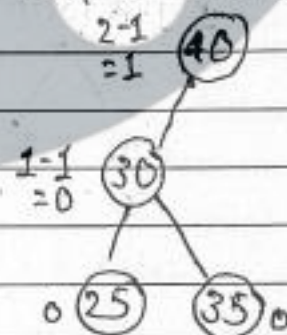
step 5:- Insert 25



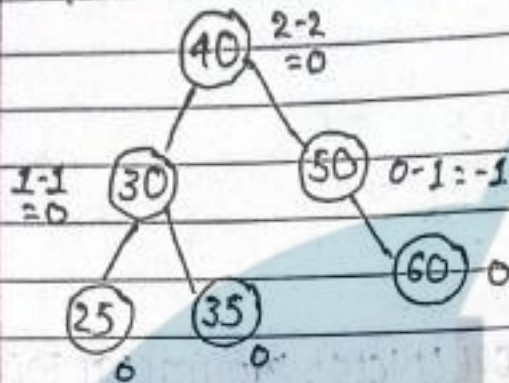
step 6: Insert 30



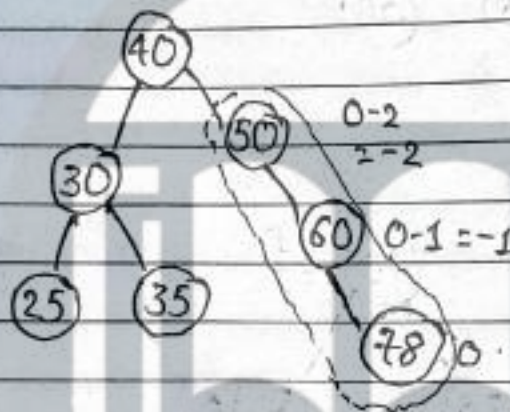
step 7: Rotation



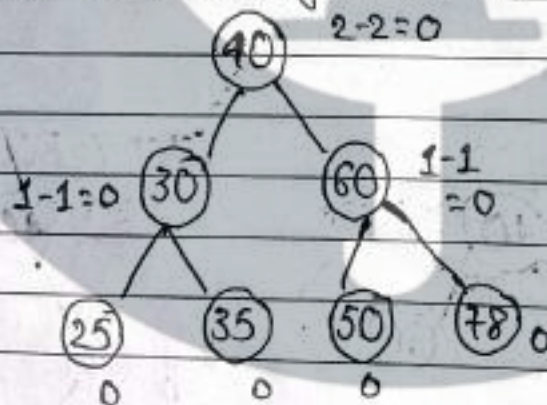
step 8:- Insert 60



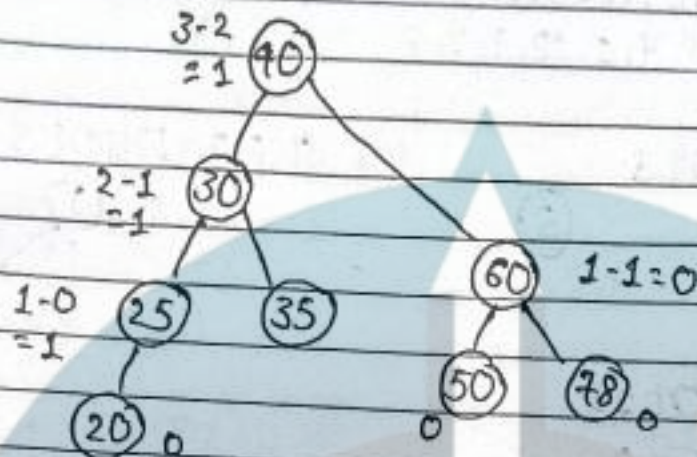
step 9:- Insert 78



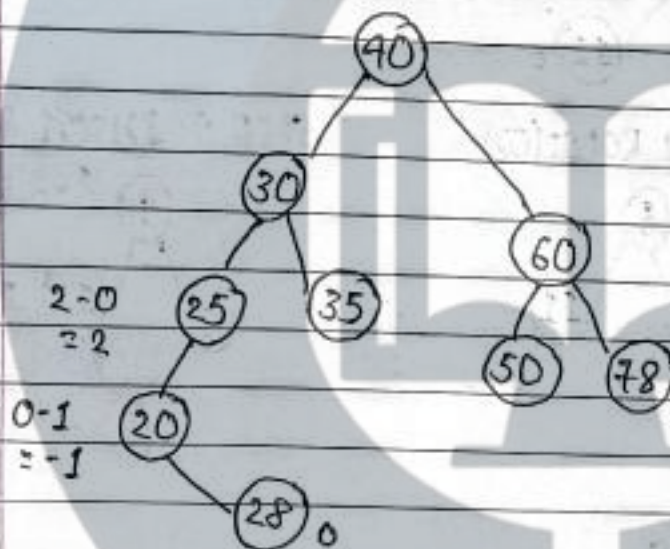
step 10:- Performing Rotation.



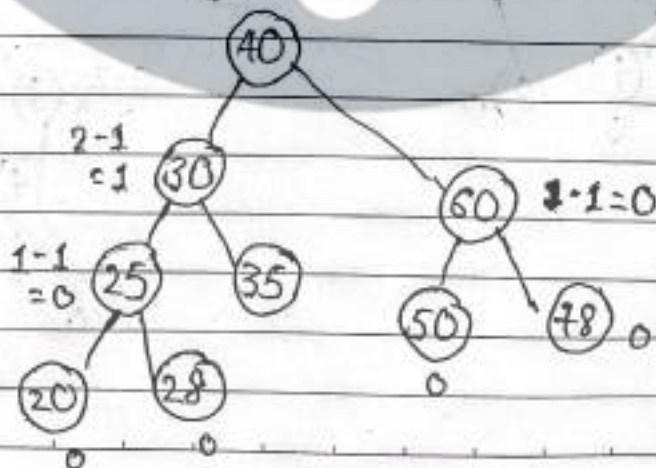
Step 11:- Insert 20



Step 12:- Insert 28



Step 13:- Performing Rotation

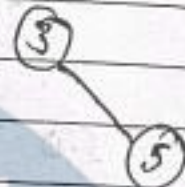


- construct AVL Tree from the following:
3, 5, 11, 8, 4, 1, 12, 9, 2, 6.

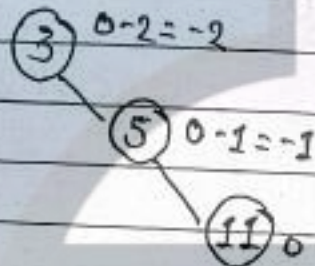
Step 1: Insert 3.



Step 2: Insert 5



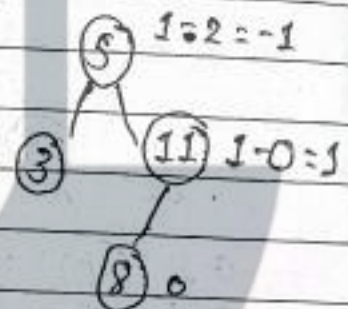
Step 3: Insert 11



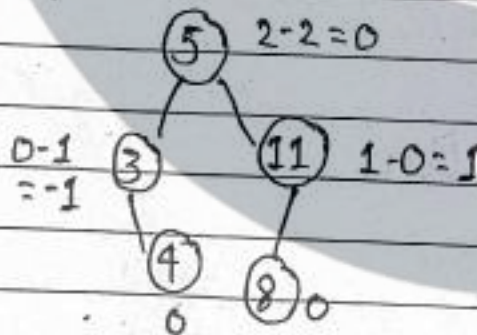
Step 4: Performing Rotation.



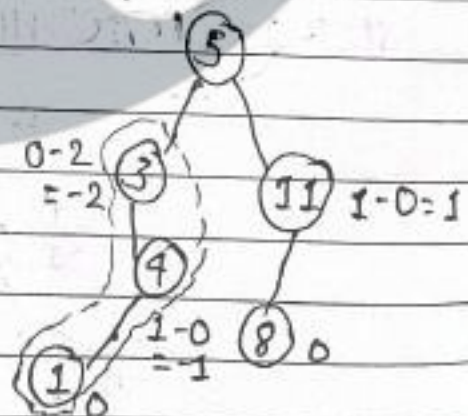
Step 5: Insert 8



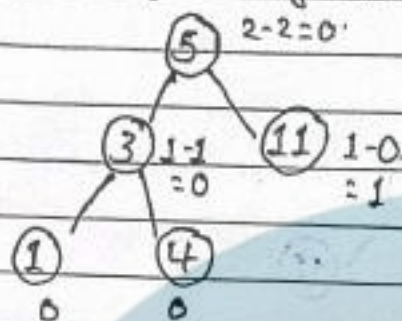
Step 6: Insert 4.



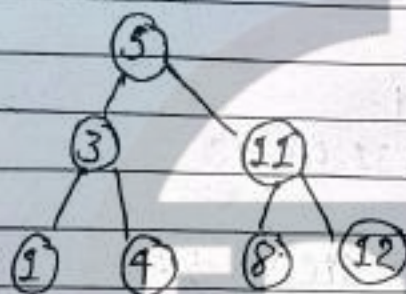
Step 7: Insert 1



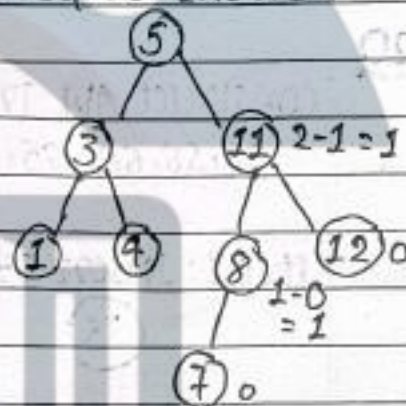
Step 8:- Performing Rotation.



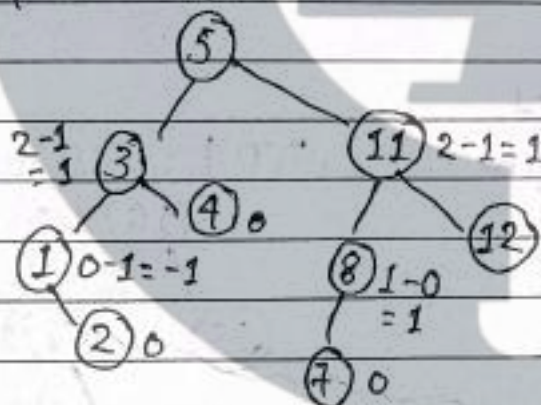
Step 9:- Insert 12.



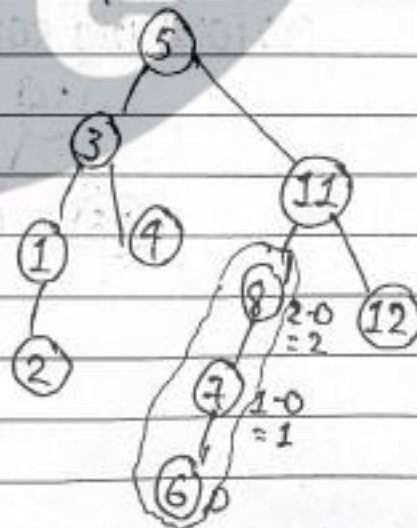
Step 10: Insert 7



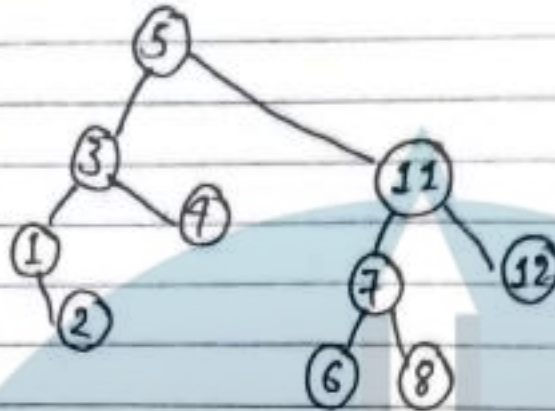
Step 11:- Insert 2.



Step 12: Insert 6



step 13: performing Rotation.



2020

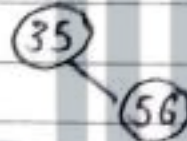
Construct AVL Tree from:

35, 56, 68, 65, 44, 41, 31, 49, 20

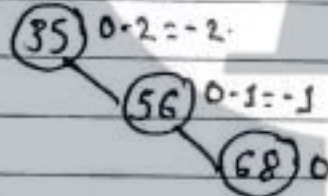
step 1: Insert 35



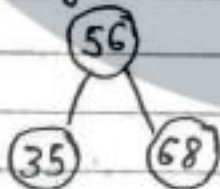
step 2: 56



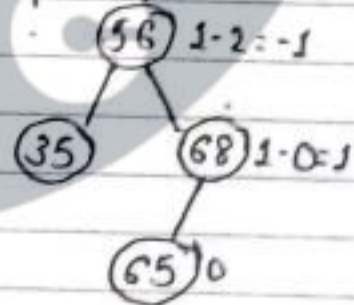
step 3: Insert 68



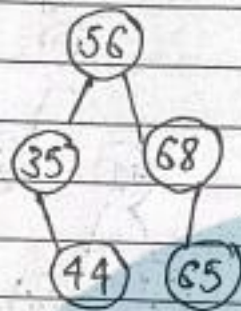
performing Rotation.



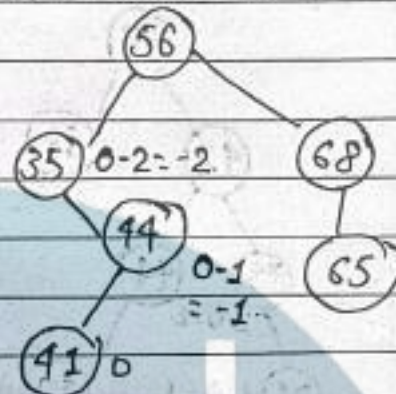
step 4: Insert 65



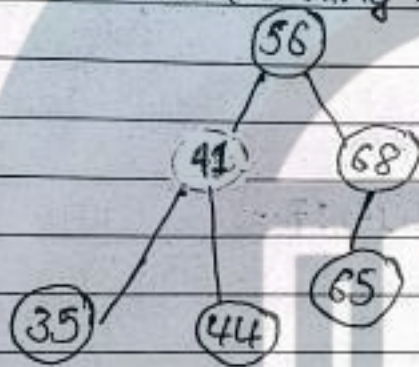
step 5: Insert 44



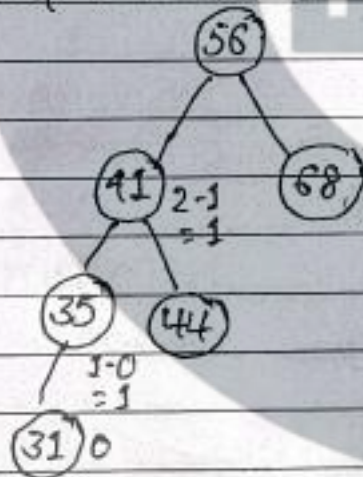
step 6: Insert 41



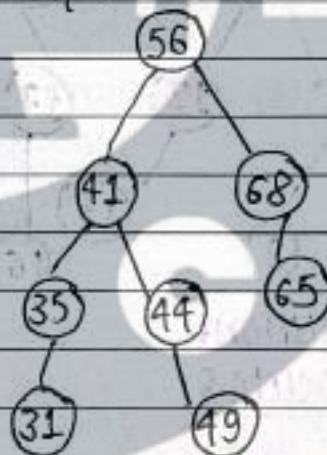
Performing Rotation:



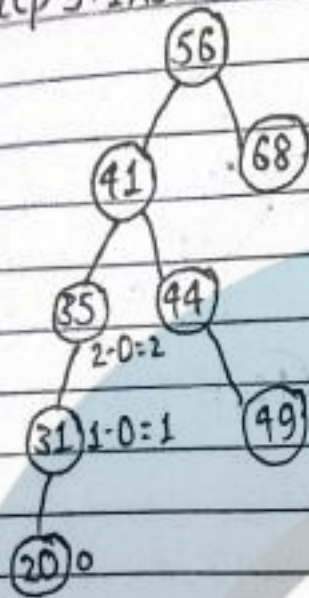
step 7:- Insert 31



step 8:- Insert 49



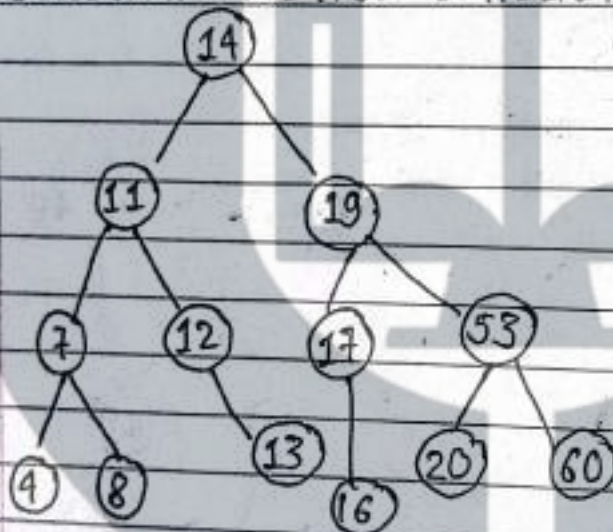
step 9: Insert 20



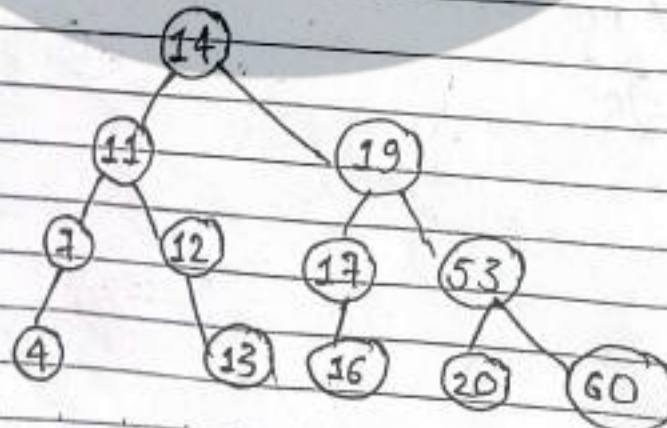
performing Rotation:-



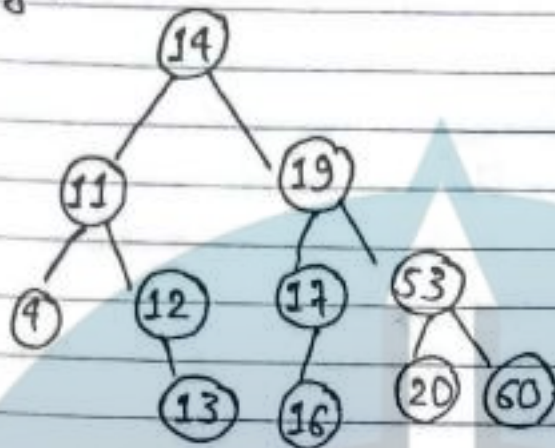
• Deletion in AVL tree: 8, 7, 11, 14, 17



Solution,
i. Delete 8



ii. Deleting 7

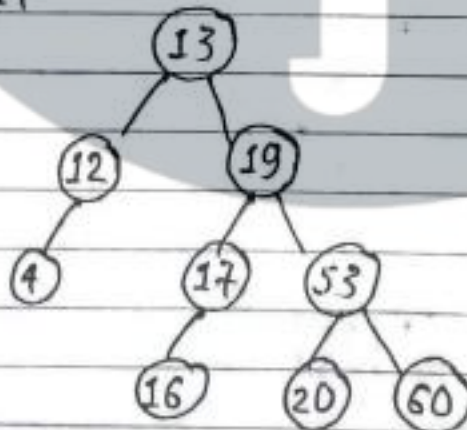


iii. Deleting 11

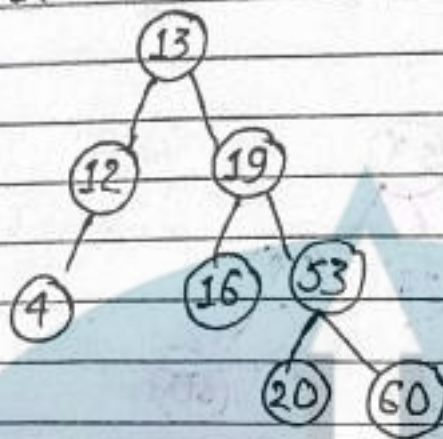


Note: While deleting 11 we can replace 11 by 12 or 4.
जुन najik छ त्यसैको ROOT Rakni.

iv. Deleting 14



v. deleting 17



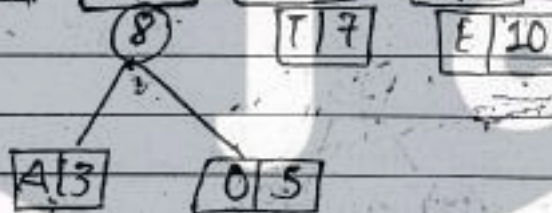
HUFFMAN Algorithm - 5 marks.

→ method for building an extended binary tree with a minimum weight path length from a set of given weights.

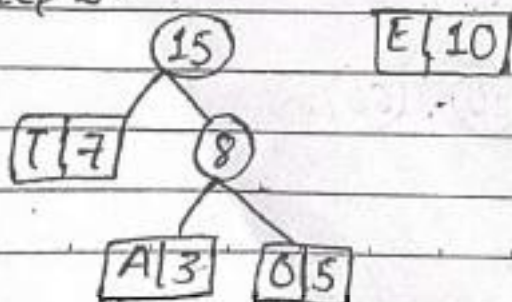
character	frequencies	code using code after diagram
A	03	110 (3 bit char)
T	07	10 (2 bit char)
E	10	0 (1 bit char)
O	05	111 (3 bit char)

Solution: A|3 T|7 E|10 O|5

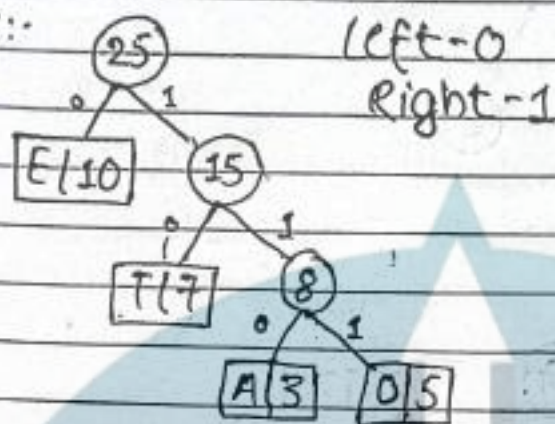
Step 1:-



Step 2:



Step 3:-



A → 1 1 0 (Yesma 3 ota Bit cha)

E → 0 (Yesma 1 ota Bit cha)

T → 1 0

O → 1 1 1

After using Huff man algorithm the total bit required is:

$$= 3 \times 3 + 7 \times 2 + 10 \times 1 + 5 \times 3$$

$$= 9 + 14 + 10 + 15$$

$$= 48 \text{ bits}$$

Tree:- is a non-linear data structure in which items are arranged in a sorted sequences. It is used to represent hierarchical relationship existing amongst several data items.

Root:- specially designed data item in a tree. It is the first in the hierarchical arrangement of data items.

Node:- Each data item in a tree is node.

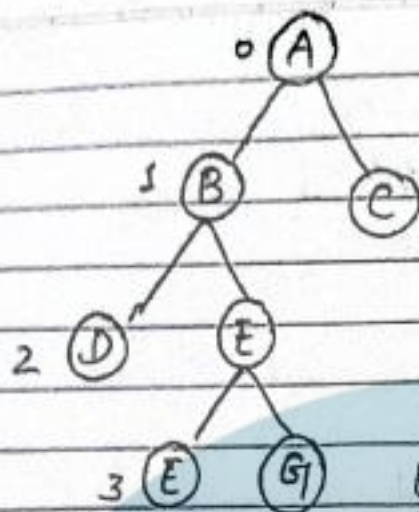


Fig: Tree.

Degree of tree is the maximum degree of node is given table. There is 3 degree.

A node with degree zero is called terminal node or leaf, such as D, F, G, C in above tree.

Any node (except the root node) which degree is not zero is non-terminal node B and E.

The children node of a given parent node are called siblings. D and E are sibling of B. They are also called brother node.

• Transversal of a Binary Tree → IMP:-

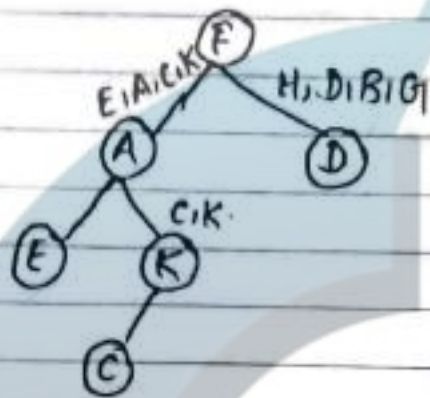
→ is a way in which each node in the tree is visited exactly once in a systematic manner. While traversing a tree root is denoted by 'N' left subtree as 'L'. Then there will be six combination of transversal i.e. NRL, NLR, LRN, LNR, RLN and RNL.

NLR → Pre-ordered traversal.

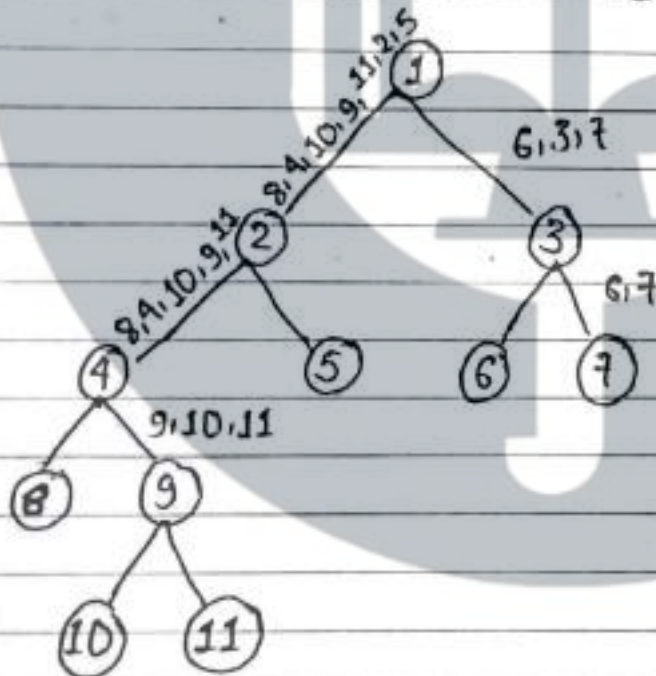
LRN → Post ordered traversal.

LNR → Inorder.

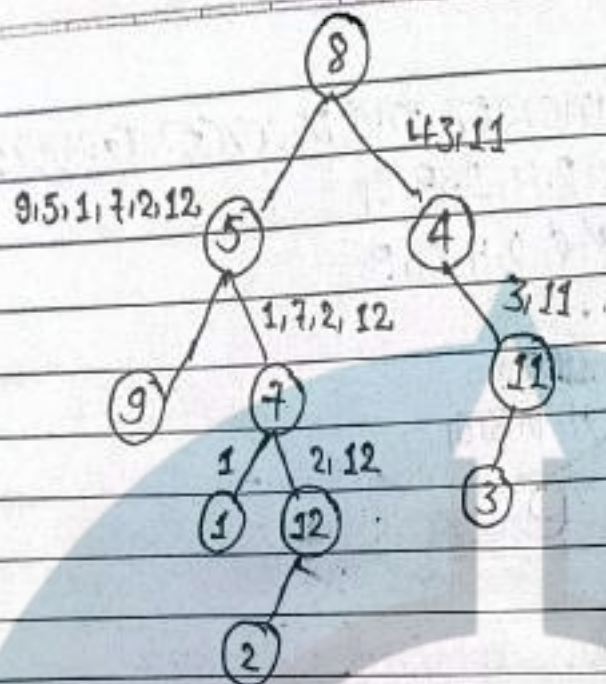
- draw a tree using inorder and preorder traversal:
 → Inorder: $E, A, C, K, F, H, D, B, G$
 → Preorder: $F, A, E, K, C, D, H, G, B$



- construction a Binary Tree from pre-order and inorder
 → Preorder: $1, 2, 4, 8, 9, 10, 11, 3, 6, 7$ (NLR)
 → Inorder: $8, 4, 10, 9, 11, 2, 5, 1, 6, 3, 7$ (LNR)



- Post order: $9, 1, 2, 12, 7, 5, 3, 11, 4, 8$ (LRN)
 → Inorder: $9, 5, 1, 7, 2, 12, 8, 4, 3, 11$



• ~~B-Tre~~ B-Tree (5 marks)

- balanced m-way tree.
- Generalization of BST in which a node can have more than one key or more than 2 children.
- Maintained sorted data.
- All leaf node must be at same level B-Tree of order m has following properties:-

• Every node has max m children.

Min children = leaf $\rightarrow 0$

Root = 2.

• Construct B-Tree:-

7, 15, 3, 16, 5, 1, 18, 9, 35, 28, 39, 17, 14, 20, 22, 25, 27 having degree order = 5.

→ solution,

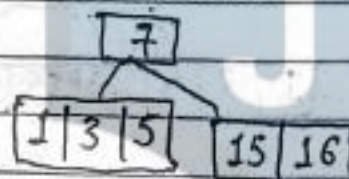
Maximum child = 5

Maximum Key = $5 - 1 = 4$.

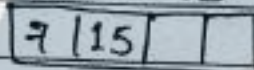
1: Insert 7



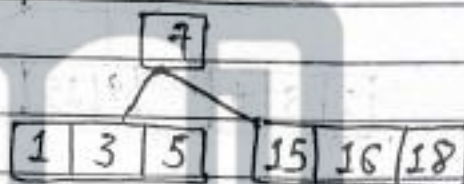
6: Insert 1



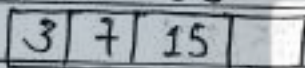
2: Insert 15



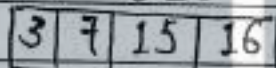
7: Insert 18



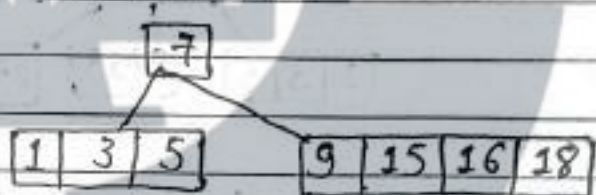
3: Insert 3



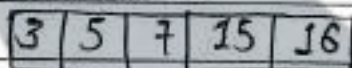
4: Insert 16



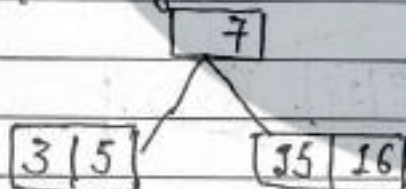
8: Insert 9



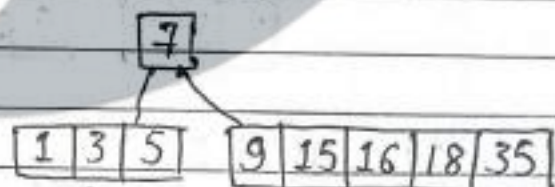
5: Insert 5



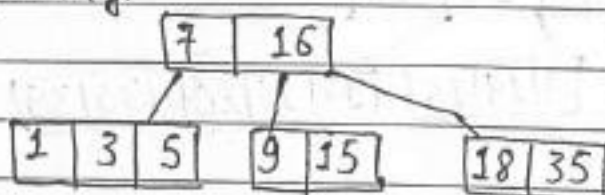
splitting:



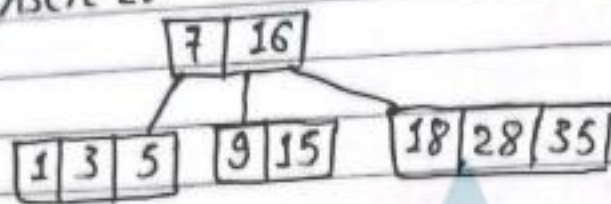
9: Insert 35:



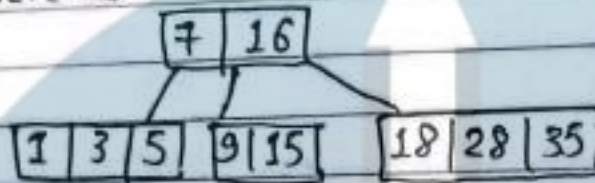
splitting,



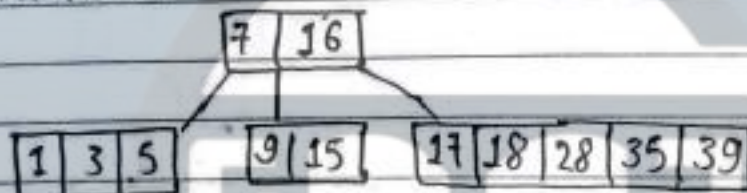
10: Insert 29



11: Insert 29



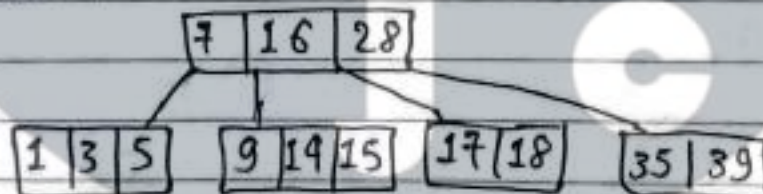
12: Insert 17



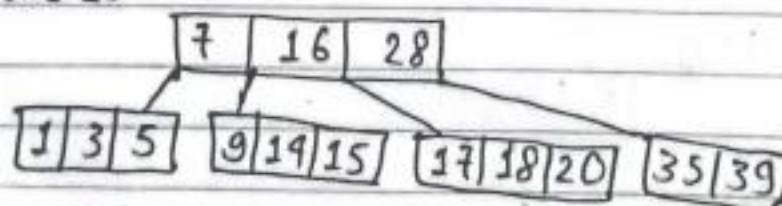
splitting,



13: Insert 14

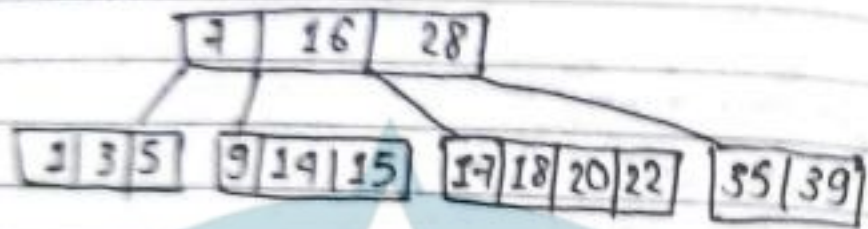


14: Insert 20

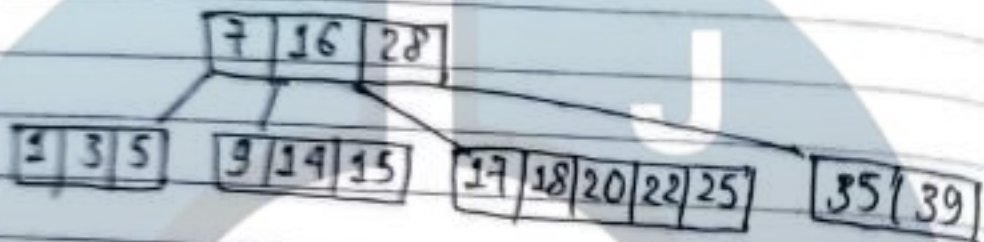


continued to B-TREE

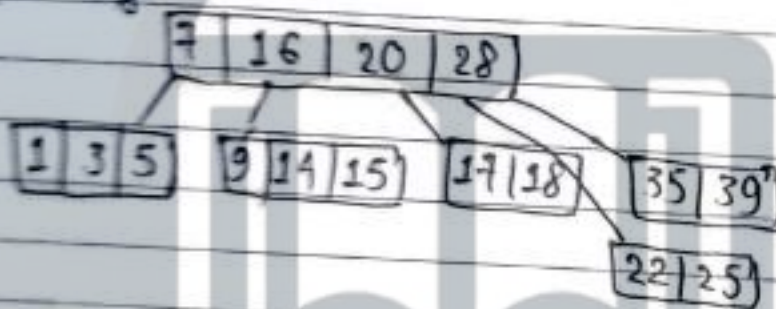
15: Insert 22



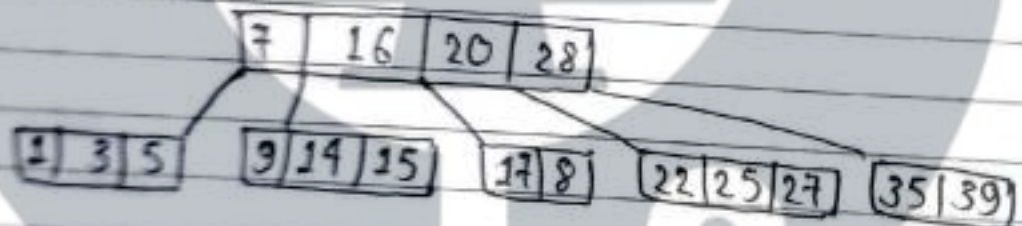
16: Insert 25



splitting,

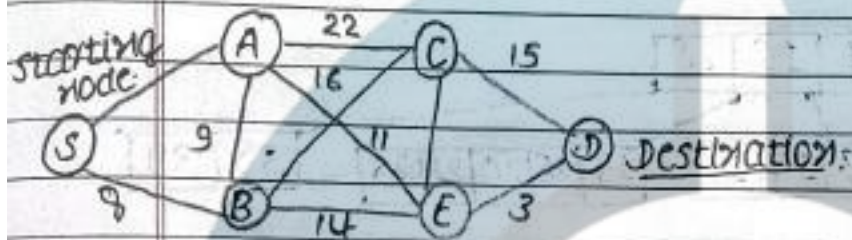


17: Insert 27

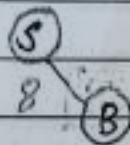


Prim's Algorithm:-

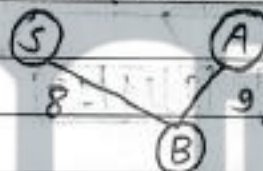
→ is a greedy algorithm like Kruskal algorithm. The algorithm always starts with a single node and moves through several adjacent nodes in order to explore all the connected edges along the way.



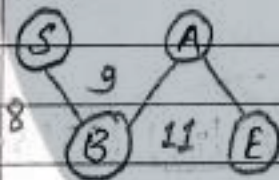
step 1:-



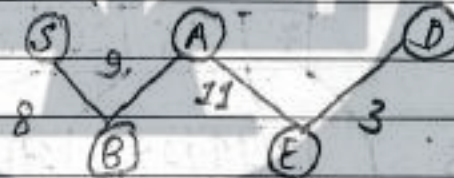
step 2:-



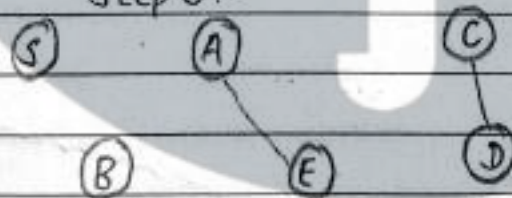
step 3:-



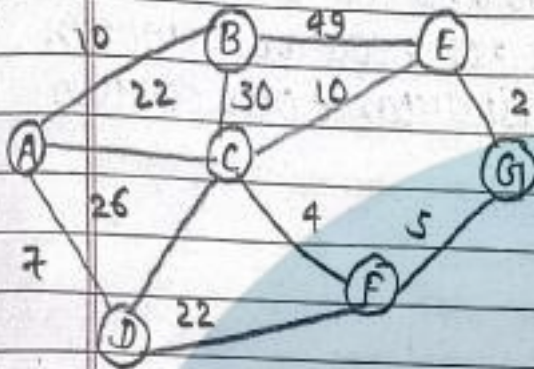
step 4:-



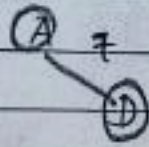
step 5:-



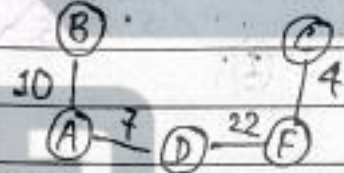
Prims Algorithm:- (find first minimum spanning tree)
- Ascending order (insert)



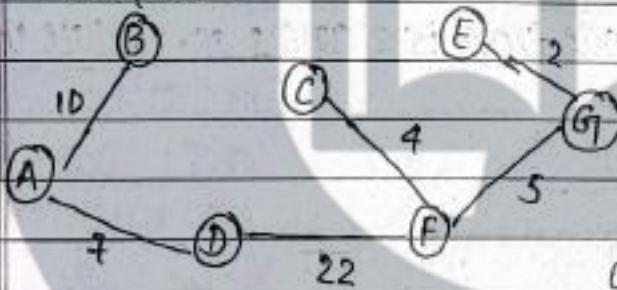
Step 1:-



Step 2:-

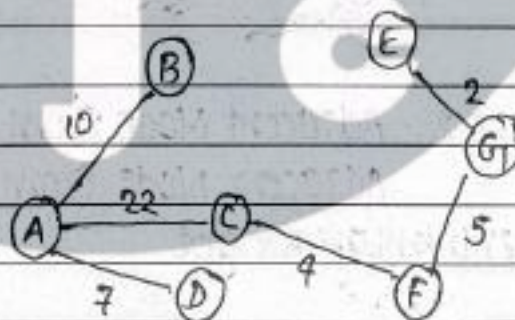


Step 3:-



OR,

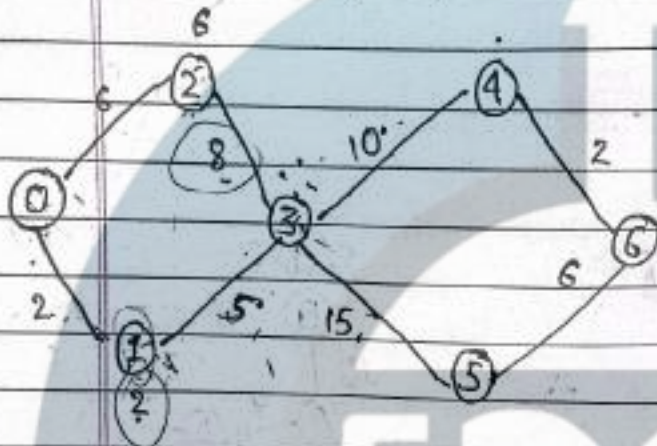
MST = 50



∴ MST = 50

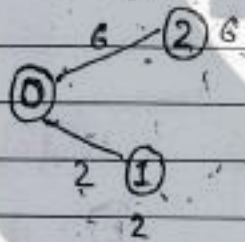
Dijkstra's Algorithm:-

→ is a popular algorithm for solving many single source shortest path problem having non-negative weight in the graph. It is to find shortest distance between two vertices in a group.

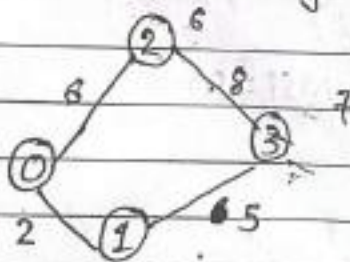


So, the shortest distance from the source vertex is 19.

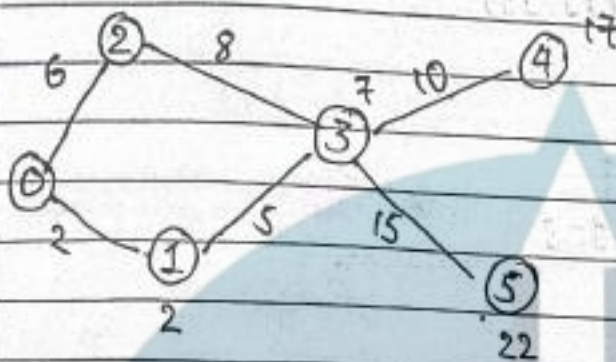
Step 1:- Adjacent Node from 0 is 2 and 1 so, update the weight.



Step 2:- Adjacent Node from 2 is 3 and Adjacent Node from 1 is 3. So update with the minimum weight.



step 3:- Adjacent node from node 3 to 4 and 5

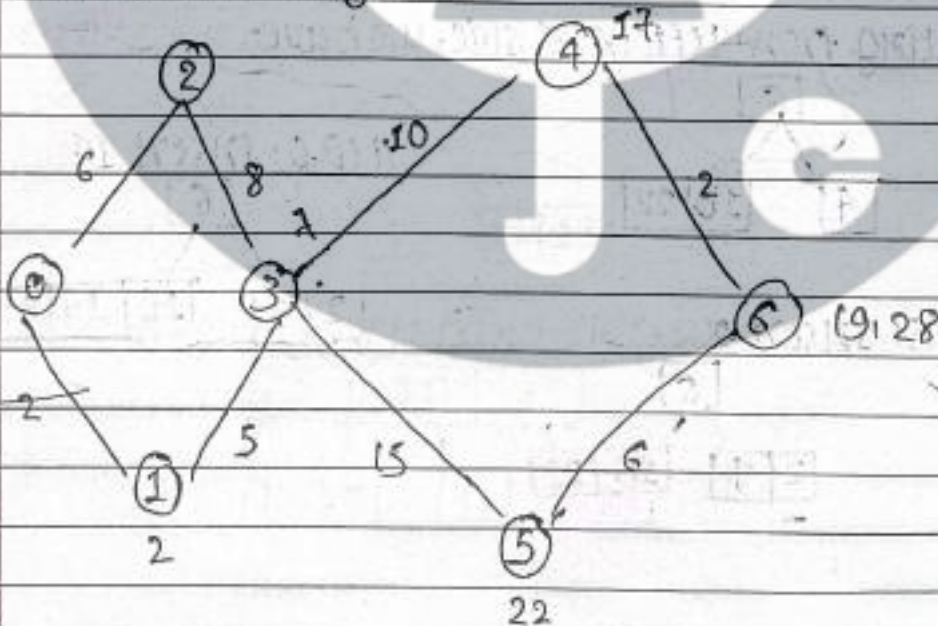


step 4:-

Ans:-



Adjacent node from node 4 is 6 and to so adjacent node from node 6 is 5 to so update with the minimum weight.



B-Tree:-

• Construct B-Tree of order 4. (2018)

6, 4, 22, 10, 2, 14, 3, 8, 11, 13, 5, 9.

Solution;

Maximum child = 4

Minimum key = $4 - 1 = 3$

Step 1:- Insert 6

[6]

Step 2: Insert 4

[4 | 6]

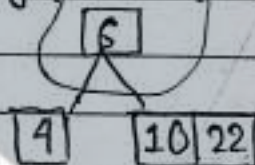
Step 3: Insert 22

[4 | 6 | 22]

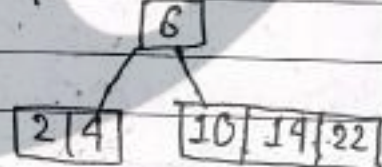
Step 4: Insert 10

[4 | 6 | 10 | 22]

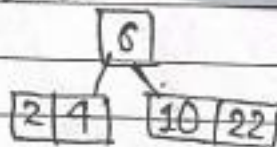
Splitting from left hand side, we have:-



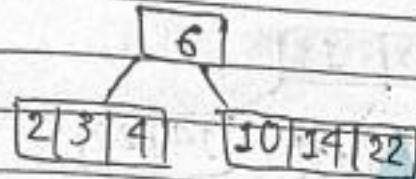
Step 6: Insert 14



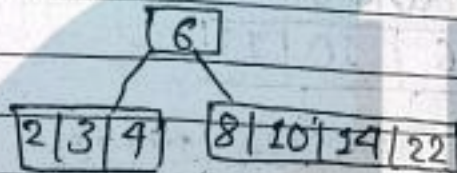
Step 5: Insert 2



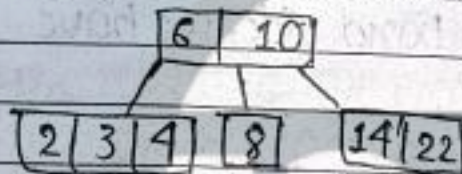
step 7: Insert 3



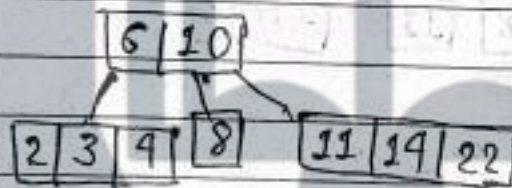
step 8: Insert 8



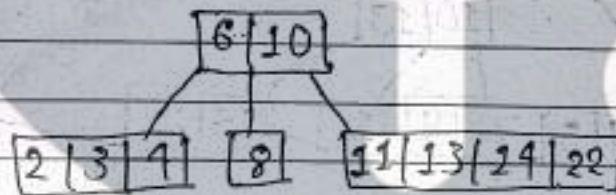
splitting from left hand side,



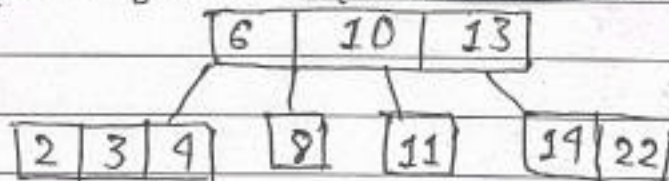
step 9: Insert 11



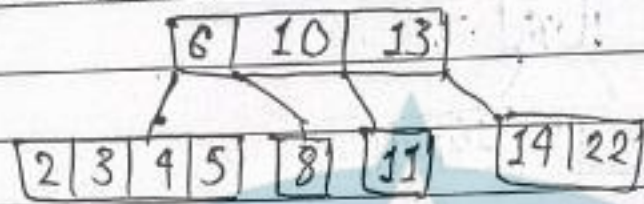
step 10:- Insert 13



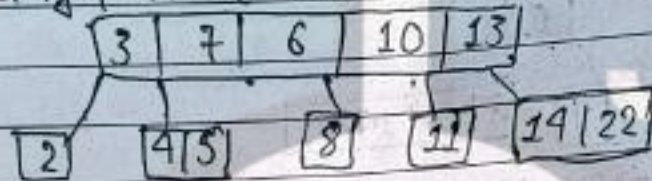
splitting from left hand side, we have:-



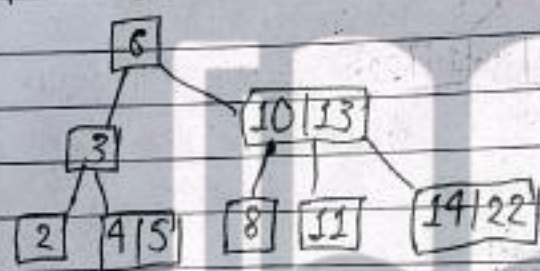
Step 11: Insert 5



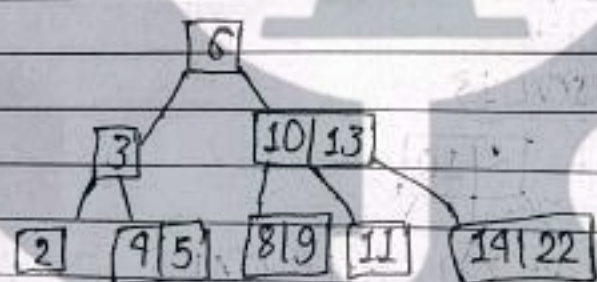
splitting from left hand side, we have:



splitting from left hand side, we have:

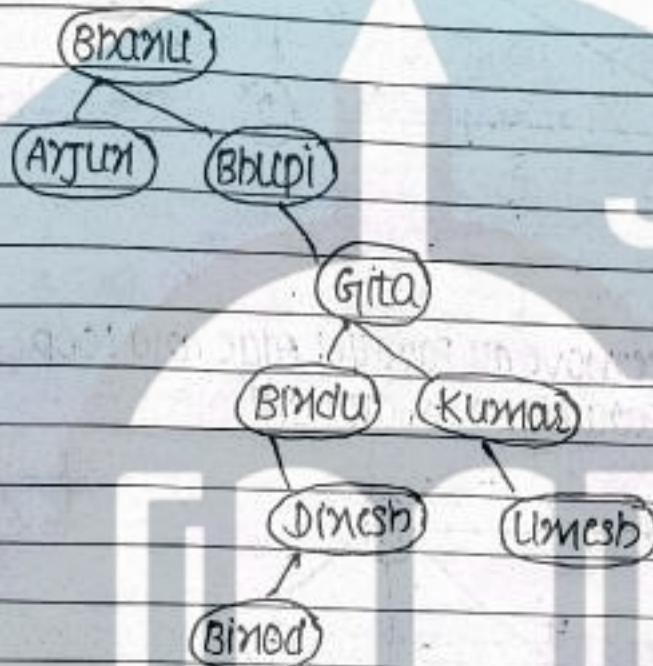


Step 12:- Insert 9



- Construct BST from the following:-

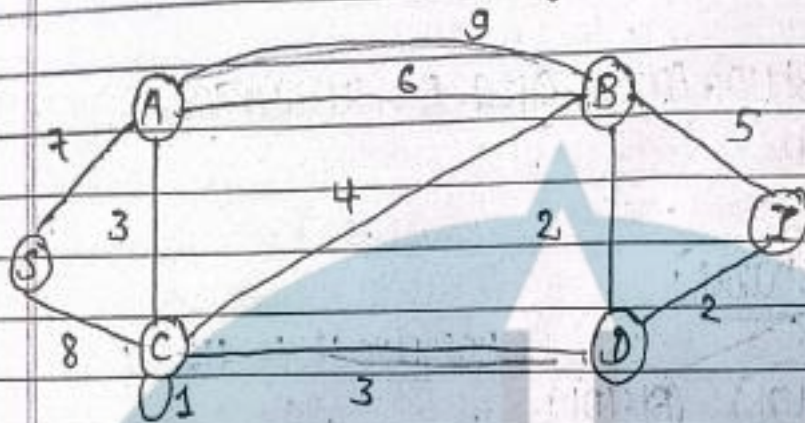
Words:- Bhanu, Bhupi, Arjun, Gita, Bindu, Kumar, Dinesh, Binod, Umesh.



- Advantages:-
 - i. Searching is effective.
 - ii. It is fast in insertion and deletion etc, when balance.
 - iii. Its code is easier as compare to link list.
- Disadvantages:-
 - i. The shape of the tree depends upon the order of insertion and it can degenerate.
 - ii. Searching operation take a longer time.

- weight Anusar Ascending order m
insert stage.

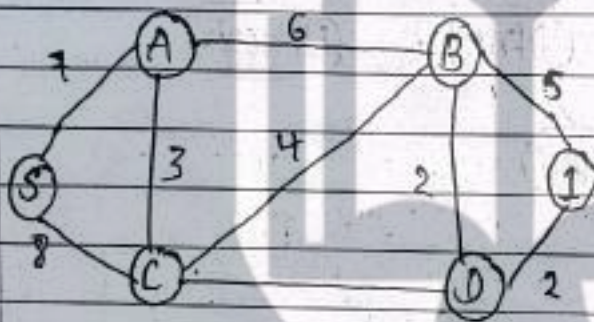
Kruskal's Algorithm;



Solution;

Step 1: Remove all Parallel edge and Loop

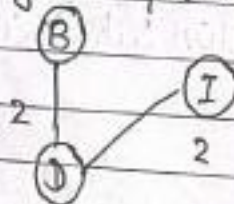
Note:- Parallel m small ligni



Step 2: Arranging in the ascending order.

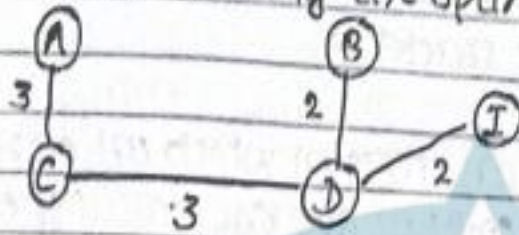
BD	TD	AC	CD	BC	BT	AB	AS	SC
2	2	3	3	4	5	6	7	8

Step 3: constructing the spanning tree of BD and TD.



classmate
Date _____
Page _____

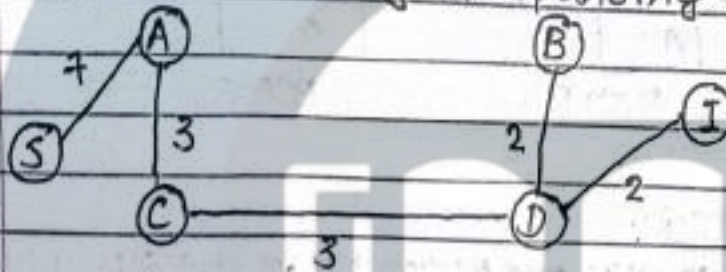
step 4: constructing the spanning tree for AC and CD.



step 5: Removing BC and BE because it create the cycle.

step 6: Removing AB because it creates cycle.

step 7: constructing the spanning tree for AS.

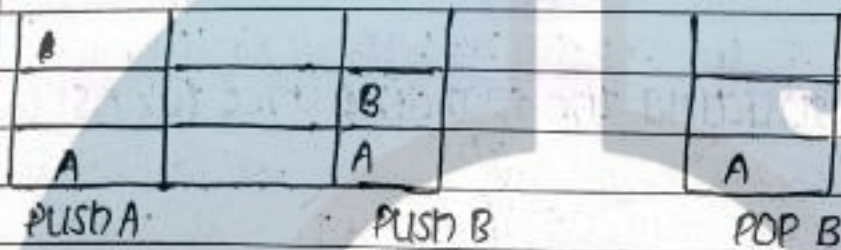


step 8: Removing SC because it create cycle.

NOTE:- MAST= Minimum cost spanning Tree.

chapter- TWO 'The stack'

A stack is linear data structure in which an element may be inserted or deleted only at one end called the top end of the stack. The stack follows the principle of last-in-first out (LIFO) system.



Application of stack:-

stack is used directly and indirectly in the following fields:-

- To evaluate the expressions (postfix, prefix).
- To keep the page-visited history in web browser.
- Used in recursion.
- To pass the parameter between the function in a C program.
- can be used as a component of other data structure.

Different operation perform on stack;

- Create Empty Stack (S)
- Push (S, X)
- Top
- Pop
- Is full
- Is empty
- Traverse-print the entire stack

condition:- for stack full (stack overflow).

If $(top == maxsize - 1)$

printf ("stack overflow").

condition for stack empty

If $(top == -1)$

printf ("stack under flow").

• Algorithm for push operation into stack (imp).

i. check whether stack is full or not.

ii. e. $top == maxsize - 1$ then stack is full.

ii. If it is full, print "stack is full".

Else,

a. Read item's value from user.

b. Increase the top by 1.

c. Push an item to top of stack.

d. For new push operation go to step (iii).

Another method of algorithm:-

if $(top == (max - 1))$

printf ("stack overflow");

else {

top = top + 1;

stack-arr[top] = pushed-item;

• Pop operation Algorithm:-

1. Start

2. If $(top == -1)$

write stack is empty or "stack else, underflow" go to step-5.

3. set value $\leftarrow$ stack[top].

4. set $TOP \leftarrow TOP - 1$

5. END

Another way:-

If ($top == -1$)

printf("stack underflow");

Else

printf("Popped element is %d", stack-arr[top]);

$top = top - 1$;

};

Stack implementation:-

i. Array implementation

ii. Linked list

Array implement:- In an array, we can push element and we can pop it. When there is no place for adding the element in array, then this is called stack overflow.

overflow is when there is no element in the stack, then value of top will be -1, so we check the value of top before deleting the element of stack.

Procedure

1. (remainder) \$power, $\wedge - 5$

* (mul), / (div) - 4

+ (add), - (sub) - 3

(- 2

/ - 1

• conversion of infix to postfix:-

Algorithm:-

- i. Add a unique symbol # in to stack and at the end of array infix.
 - ii. scan symbol of array infix from left to right.
 - iii. If the symbol is left parenthesis '(' then add it in to the stack.
 - iv. If the symbol is operand then add it to array post fix.
 - v. a. If symbol is operator then pop the operator which have same or higher precedence then operator which occur.
b. Add the popped operator to array post fix.
c. Add the scanned symbol operator in to stack.
d. If symbol is 'right parenthesis ')' then pop all the operator from stack until left parenthesis '(' from stack.
 - e. If symbol is # then pop all the symbol from stack and add them to array post fix except #.
- do the same process until # comes in scanning array infix.

change infix to postfix:

$a+b \times c-d/e \times f$

Token	operator stack	postfix string
a	#	a
+	+	a
b	+	ab
*	+*	ab
c	+*	abc
-	-	abc*+
d	-	abc*+d
/	-/	abc*+d
e	-/	abc*+de
*	-*	abc*+de
f	-*	abc*+def
	#-	abc*+def*
	#	abc*+def*-

$A+(B \times C + (D/E - F) \times G) \times H$

Token	operator stack	postfix string
A	-	A
+	+	A
C	+C	A
B	+C	AB
*	+C*	ABC
C	+C*	ABC*
-	+C-	ABC*
(	+(-	ABC*
D	+(-D	ABC*D
	+(-C	ABC*D

I	+ (- (/	ABC * DE
E	+ (- (* /	ABC * DE /
-	+ (- (-	ABC * DE / E
F	+ (- (-	ABC * DE / F
)	+ (-	ABC * DE / F -
*	+ (-	ABC * DE / F -
G	+ (- *	ABC * DE / F - G
)	+ (- *	ABC * DE / F - G * -
*	+ (- * +	ABC * DE / F - G * -
H	+ (- * +	ABC * DE / F - G * - H
-	+ (- * +	ABC * DE / F - G * - H * +

converting Infix Expression to Prefix Expression :-

Algorithm:-

1. Reverse the input string.
2. Examine the next element in the input.
3. If it is operand, add it to the output string.
4. If it is closing parenthesis, push it on stack.
5. If it is an operator, then.
 - (i) if stack is empty, push operations on stack.
 - (ii) if top of the stack is closing parenthesis push operator is on stack.
 - (iii) if it has same or higher priority then top of the stack, push operator on s.
 - (iv) else pop the operator from the stack and add it to output string, repeat 5.
6. If it is opening parenthesis, pop operator from stack and add them to s until a closing parenthesis is

encountered POP and discard the closing parenthesis.

7. If there is more input go to step 2:

8. If there is no more input, unstack the remaining operators and add them.

9. Reverse the output string.

• Infix to Prefix Expression:-

i) $A + (B * C - (D / E \wedge F)) * G$

→ First, reverse the given string.

$(G * (F \wedge E / D) - C * B) + A$

Now,

Character	Stack	Postfix
(	(	-
G	(	G
*	(*	G
C	(*C	G
F	(*C	GF
^	(*C^	GF^
E	(*C^E	GFEE
/	(*C^E/	GFEE^
D	(*C^E/D	GFEE^D/
-	(-(*C^E/D/	GFEE^D/^
C	(-(*C^E/D/^C	GFEE^D/^C
*	(-(*C^E/D/^C*	GFEE^D/^C*
B	(-(*C^E/D/^C*B	GFEE^D/^C*B
)	(-(*C^E/D/^C*B)	GFEE^D/^C*B)
+	(+(-(*C^E/D/^C*B)	GFEE^D/^C*B+
A	(+A	GFEE^D/^C*B+A
-	-	GFEE^D/^C*B+A+

Prefix = $+A-*BCT/D^{\wedge}EFG$

(ii) $(A/B \wedge C + D)$

→ Reversing, $(D + C \wedge B / A)$

Symbol	Stack	Postfix
(	(	.
D	(D	D
+	(+	D
C	(+C	DC
^	(+^	DC
B	(+^B	DCB
/	(+ /	DCB^
A	(+ /A	DCB^A
)	(+ /)	DCB^A
-	.	DCB^A / +

Prefix: $+ / A^B C D$

- Evaluation of a postfix expression (cmp).

Algorithm:-

- scan 1 character at a time from left to right.
- If scanned character is operand then push it to the stack.
- If scanned character is operator then POP two operands from stack, first one popped is operand 2 and second one is operand 1.
- Perform result = (operand 1) operator (operand 2) and push the result into stack.
- Repeat above steps till end of the input.

1. Evaluate following postfix expression:-

: AB+C*DE--FG+\$.

Where, A=1, B=2, C=3, D=4, E=3, F=2 and G=1.

S.N.	scan character	Operand 1	Operand 2	Stack
1	A(1)			1
2	B(2)			1, 2
3	+	1	2	3
4	C(3)			3, 3
5	*	3	3	9
6	D(4)			9, 4
7	E(3)			9, 4, 3
8	-	4	3	9, 1
9	-	9	1	8
10	F(2)			8, 2
11	G(1)			8, 2, 1
12	+	2	1	8, 3
13	\$	8	3	5, 12

∴ Required value of given expression is 512.

Postfix Expression:-

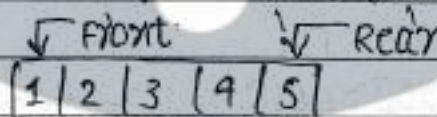
5 4 6 + * 4 9 3 / + *

character	Operand 1	Operand 2	stack
5			5
-4			5, 4
6			5, 4, 6
+	4	6	5, 10
*	5	10	50
4			50, 4
9			50, 4, 9
3			50, 4, 9, 3
/	9	3	50, 4, 3
+	4	3	50, 7
*	50	7	350

∴ The require value of the postfix expression is 350.

Chapter:- 6

• Queues:- is a non-primitive linear data structure. It is a homogenous collection of elements in which new elements ~~in which new elements~~ are added at one end called the Rear end, and the existing elements are deleted from other end called front end first-in-first out property.



• Queue as an ADT:-

(i) Make Empty (q) = to make q as an empty queue.

(ii) Is empty (q) = check q is empty, if empty returns true or false.

(iii) Is full (q) = check q is full, if full return true or

False.

iv) Enqueue (q, x): insert at rear of the queue, if and only if q is not full.

v) Dequeue (q): Delete items from front of the queue, if q is not empty.

vi) Traverse (q): To read entire queue that is display the content of the queue.

• primitive operation in a queue:-

1) Enqueue

→ Adds an element at the beginning of the queue.

Algorithm:-

i) Declare and initialize necessary variable such as maxsize, Rear, front, queue [Rear], data.

ii) If (rear = maxsize - 1) {

printf ("queue overflow"); }

else if (front == -1) {

front = 0;

Rear = 0; }

else {

Rear = Rear + 1;

queue [Rear] = data;

}

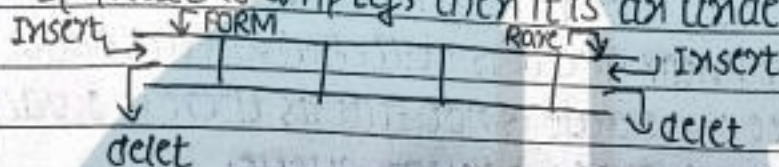
iii) For next enque go to step 2.

2. Dequeue

→ stands for double end queue.

Insertion and deletion operation can be done at both FRONT and END of the queue.

- If queue is empty, then it is an underflow.



Algorithm:-

1. Declare and initialize necessary variables such as FRONT, REAR, item.

2. If $(\text{FRONT} == -1)$ {

$\text{printf}(\text{"Queue underflow"});$ };

 else if $(\text{FRONT} == \text{REAR})$ {

$\text{FRONT} = -1;$

$\text{REAR} = -1;$

 }

 else {

$\text{item} = \text{queue}[\text{FRONT}]$

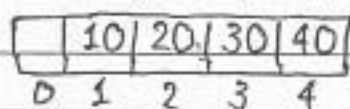
$\text{FRONT} = \text{FRONT} + 1$

$\text{printf}(\text{"The delete is : "}, \text{item});$

 }

3. for next dequeue operation go to step 2.

* Limitation of Linear Queue:-



Linear elements in sequential manner
Not " - FIFO (Both)
sufficient
No order:
(insert - Rear) insufficient (memory space).
Delete - Front). FIFO

- In linear queue first and rear indices are always increase but never decreases as the items are removed (dequeued) from the linear queue, the vacant space can't be utilize or used.

Also, from above linear queue we see that value of rear is 4. i.e. value of $\text{rear} = \text{maxsize} - 1$ which is the condition for linear queue is full. But, in the above linear queue we see the queue is not full as there is 1 vacant space at beginning of the linear queue.

So, to overcome these limitation we need circular queue.

- circular queue: It is also a data structure which connect the two end of a linear queue and work on the basis of FIFO. In this queue and array are view as circular instead straight. It is also called operation in circular queue.

- Insertion (Enqueue): - insert element
- Deletion (Dequeue): - Delet



- Types of implementation of queue:-

1. Linked list implementation
2. Array implementation, Types:-
 - i) Linear
 - ii) Circular
 - iii) Priority.

→ IMP

Algorithm for the enqueue and dequeue operation:-
in a circular queue.

1. Declare and initialize necessary variables such as
 $rear = 0$, $front = 0$ item, $maxsize$, $queue [maxsize]$

2. For enqueue operation:-

i) IF $(front == (rear + 1) \% maxsize)$
 write queue overflow and exit.

ii) Else: Read an item from the user
 $queue [rear] = item$;
 $rear = (rear + 1) \% maxsize$.

3. For next enqueue operation go to step 2.

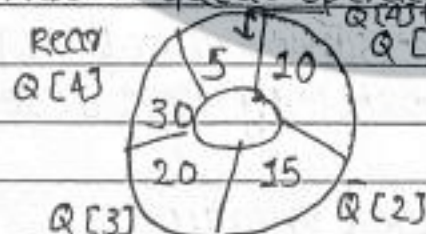
4. For dequeue operation:-

i) IF $(rear == front)$
 Print Queue is underflow and exit.

ii) Else

item = $queue [front]$;
 $front = (front + 1) \% maxsize$.
 Display an item.

5. For next dequeue operation go to step 4.



For insertion

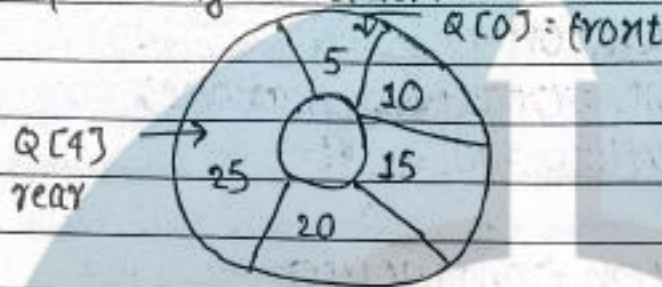
$rear = (rear + 1) \% maxsize$

$rear = (4 + 1) \% 5$

= 0

rear = front so the overflow condition satisfy.

- Explaining deletion:-



Application of circular queue:-

- Memory Management
- CPU-scheduling
- Traffic system.

front = (front + 1) % maxsize

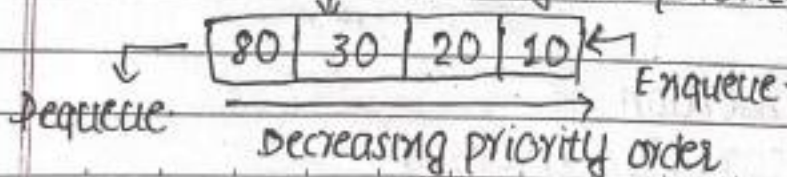
If we delete the element from queue it will delete front, front and front should be modified.

- Priority Queue:- collection of elements where elements are stored according to their priority level. Inserting and removing of elements from queue is decided by the priority of the elements. The order in which elements are deleted and processed comes from the following rules:-

i. An element of higher priority is processed before any element of lowest priority.

ii. Two elements with the same priority are processed according to order in which they were added to queue.

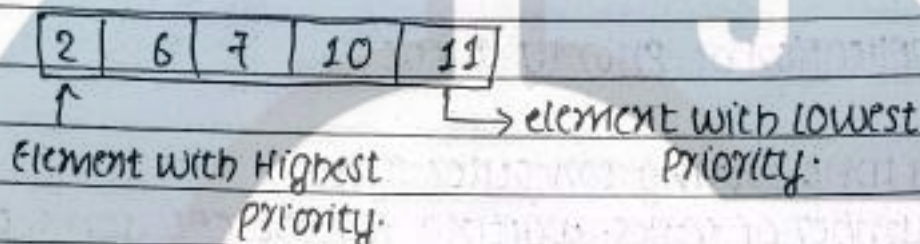
Element with highest priority:-



- Ascending priority queue:-

→ A collection of items into which item can be inserted arbitrarily and from which only the smallest items can be removed is called ascending priority queue.

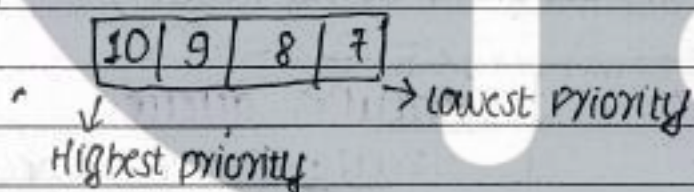
For eg:- we have taken number from 1 to 5 arranged in ascending order, therefore the smallest number i.e. 1 is given as the highest priority in a priority queue.



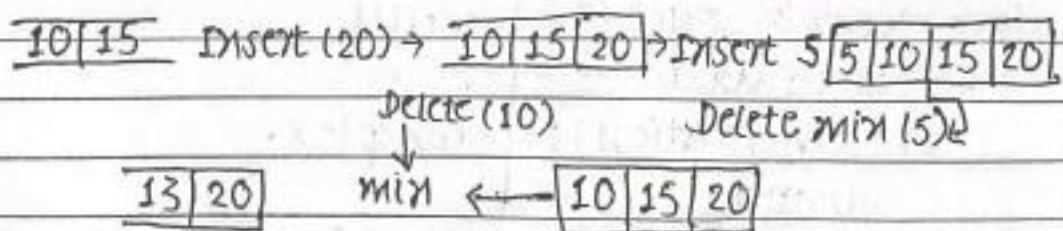
- Descending priority queue:-

→ allows only the largest items to be removed. In descending order priority queue, a higher priority number is given as a higher priority in a priority. For eg, we take the numbers from 1 to 5 arranged in descending order.

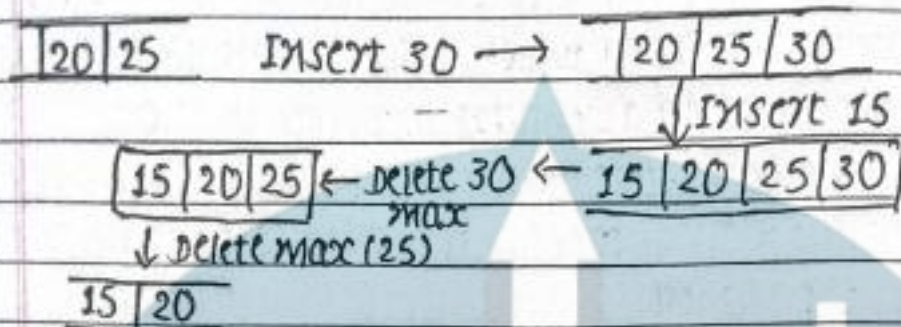
Therefore, the largest number 5 is given as the highest priority in a priority queue.



- Minimum (ascending) priority queue:-



Maximum (Descending) Priority Queue:

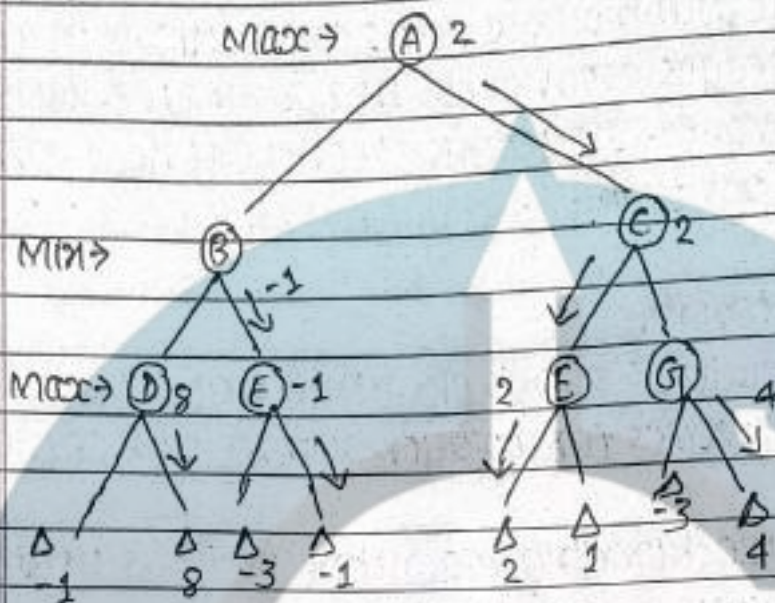


• Application of Priority Queue:-

- (i) In time sharing computer system there may be large number of tasks. waiting for the CPU. some task has higher priority and some tasks has lower priority. The set of task waiting for a CPU forms of priority queue.
- (ii) All type of customer services (like railway ticket reservation) center softwares are designed using queue to store customers information.
- (iii) Printer server are designed using queue.

Linear, queue structure straight line	Circular queue. circular (rear connect to front).
- space. Insufficient overflow occur if space is free as well.	sufficient
implementation is simple.	- full
use - Task scheduling	- complex.
	• Buffering, Round Robin

- **Game tree:-**
 - is a directed graph whose nodes are positions in a game and whose edges are moves. In a game tree, every node corresponds to a board position.
- **Minimax Theorem:-**
 - It is a specialized search algorithm that returns optimal sequence of moves for a player in zero sum game.
 - i. Recursive / Backtracking algorithm which is used in decision making and game theory.
 - ii. Use recursion to search through game tree.
 - iii. Algorithm computes minimax decision for current state.
 - iv. ~~2 types:-~~ 2 player:-
 - v. Depth first algorithm is used for exploration of complete game tree.
- **Properties of minimax:-**
 - i. Time complexity - $O(b^m)$
 - ii. Space complexity - $O(b^m)$
- **Limitation:-**
 - is very slow for complex game, such as chess, go etc, because these have branching factor. So, this limitation of minimax can be improved from
 - alpha-beta pruning.

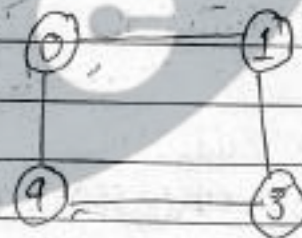
Max $\rightarrow$ 

Chapter- 10 (Graphs)

$\rightarrow$ is non-linear data structures consisting of nodes and edges. The nodes are sometimes also referred to as vertices and edges are lines or arcs that connect any 2 nodes in the graph.

In these graph, set of vertices are: $V = \{0, 1, 3, 4\}$
and set of edge:

$E = \{01, 04, 43, 13\}$



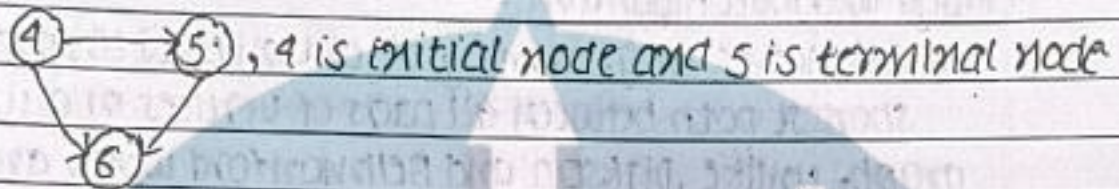
Types of graphs:-

1. Undirected graph:-

$\rightarrow$ is a graph in which edges have no orientation. Since, its edges are not attached with any of the direction. The above graph is an example of undirected graph.

2. Directed Graph:-

→ is a graph in which edges have orientation, edges from an ordered pair.



3. Directed Acyclic Graph (DAG)

4. Multigraph

5. Simple graph

6. Weighted and unweighted graph.

7. Complete graph.

Graph as an ADT:-

i. Graph () creates a new empty graph.

ii. Add Vertex adds an instance of vertex to the graph.

iii. Add Edges From Vert to Vert adds a new directed edge to the graph that connects two vertices.

iv. Add Edges (From Vert, To Vert, Weight) adds a new weighted directed edges to the graph that connected two vertices.

v. Get Vertex (Vert Key) finds vertex in the graphs.
Name Vertkey.

vi. Get Vertex List () returns the list to all the vertices in the graph.

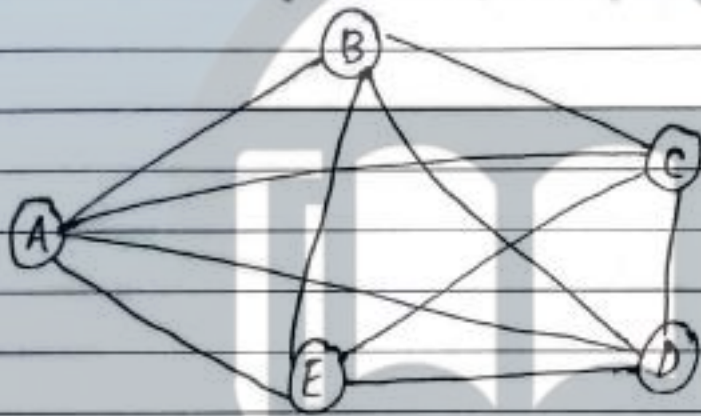
vii. Get Vertex Keys () returns a set of all the vertex keys in the graph.

viii. contains (vertex) returns true if the given

vertex in the graph, false otherwise.

- Floyd Warshall Algorithm:-

→ is a dynamic programming algorithm used to find the shortest path between all pairs of vertices in a weighted graph, unlike Dijkstra and Bellmanford which are single source shortest path algorithm. It work for both directed and undirected graph and can handle graphs with both positive and negative weight edges.



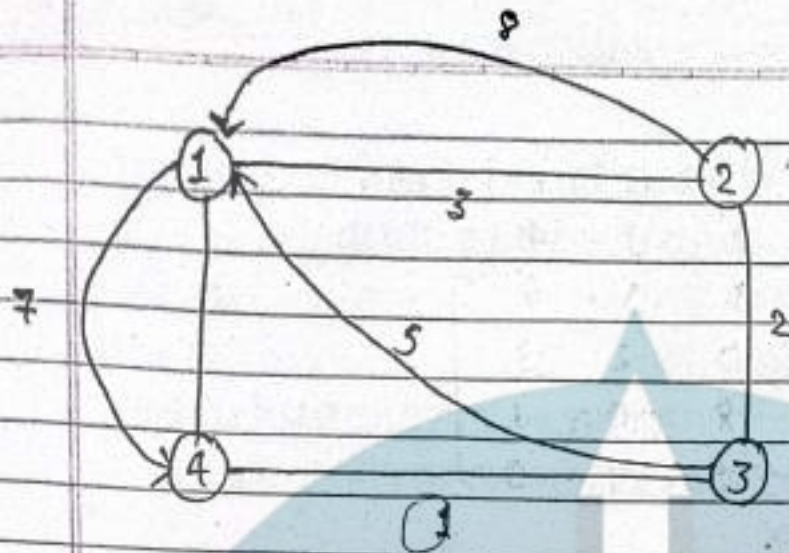
Algorithm:-

1. created a distance matrix $dist[i][j]$, where $dist[i][j]$ holds the weight of edge between node i and j . If no edge exist set it to infinity. set $dist[i][i]$ to 0.

2. for each node k acting as intermediate node for each pair of node (i, j) .

Update $dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j])$.

3. After passing all nodes, $dist[i][j]$ gives the distance from i to j .



:(Problem):-

Step 1:-

	1	2	3	4
1	0	8	5	7
2	8	0	2	∞
3	5	∞	0	1
4	7	∞	1	0

Do the first row & column same

Step 2:-

	1	2	3	4
1	0	8	5	7
2	8	0	2	15
3	5	8	0	1
4	7	5	1	0

Number ko jo xha usko row and column same & diagonal are always 0.

Step 3:-

	1	2	3	4
1	0	8	5	7
2	8	0	2	15
3	5	8	0	1
4	7	5	1	0

3-2
8
2-8
7
1-4

Wishu

Step 4:-

	1	2	3	4
1	0	3	5	6
2	7	0	2	3
3	5	8	0	1
4	2	5	7	0

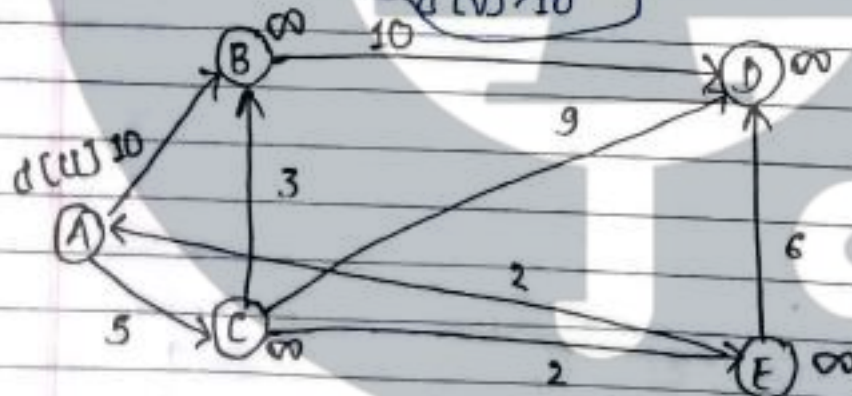
Step 5:-

	1	2	3	4
1	0	3	5	6
2	5	0	2	3
3	3	6	0	1
4	2	5	7	0

∴ shortest path from 1 to 4 is 6.

∴ shortest path from 2 to 3 is 2.

$d[u] = 0 + 10 = 10$ → NOTE:
 $d[v] > 10$



Find the shortest path if the source vertex is A. Using table
→ SOLN:- we can go A to B or C not E, But we can come from E to A; due to arrow.

Formula:- $If (d[u] + c[u,v] < d[v])$
 $d[v] = d[u] + c[u,v]$

Where,

$d[u]$: start vertex number.

$d[v]$: destination vertex number which will be ∞ at first.

$c[u,v]$: number or time to go from start to destination vertex. For eg; A to B = 8.

Note:- The array inside table are for path start vertex A.

selected vertex	A	B	C	D	E
A	[0] ←	∞	∞	∞	∞
C		10	[5] ←	∞ ←	∞ ←
E		8		14	[7] ←
B		[8]		13	← same 50
D				[13]	← escape B.

Note:- we don't write 18 in D at last because 18 is greater than 13. so we chose 13.

Thus,

The shortest path from vertex A to B = 8

" " " " " A to C = 5

" " " " " A to D = 13

" " " " " A to E = 7

A to D path

∴ DECA

∴ A C E D = 13

∴ 5 + 2 + 6 = 13

Graphs Transversal:-

- BFS use Queue.
- DFS use Queue.

Disadvantage of this algorithm

→ If there is negative value or distance in a cycle it will not give correct answer.

• Linear search (sequential search) → data uncertain में काम नहीं जहाँ।
→ is a very simple search algorithm. In this type of search a sequential search is made over all items one by one. Every item is checked and if the match is found then that particular item is returned, otherwise search continues till the end of the data collection.

Example:-

10 14 19 26 27 31 33 35 42 44

→ = 33

compare (if not equal) = no value.

Algorithm:-

- $i = 1$ & $S = 0$
- if $i > n$, go to step 7.
→ maximum size of list loop.
- if $A[i] = x$, go to step 6.
- $i = i + 1$.
- Go to step 2.
- Print element x found at i .
- Print element not found.
- Exit.

List 2							
4	5	10	12	16	14	33	42
0	1	2	3	4	5	6	7

$i = 0, x = 16$

$A[i] = A[0] = 4$

$A[i] = x$ or whether $4 = 16$

[NOT]

→ $i = 0 + 1 = 1$

$A[i] = 16x$

$i = 2, A[2] = 16, x$

Searching:-
process of finding required data from data base.

$i = 4$, $A[4] = 16$ 2, Element 16
found at A

Exit.

- Binary search:-
 - is a fast search algorithm whose runtime complexity is $O(\log n)$.
 - This search algorithm works on principle of divide and conquer.
 - For this algorithm to work properly, the data collection should be in sorted form.
 - Binary search looks for a particular item by comparing middle item of collection.
 - If a match occurs, then index of item is returned.
 - If middle item is greater than item, then item is searched in sub-array to left of the middle item.
 - Otherwise, item is searched for in the sub-array to the right of middle item.
 - This process continues on the sub-array as well until size of subarray reduces to zero.

10	14	19	26	27	31	33	35	42	44
0	1	2	3	4	5	6	7	8	9

$$\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$$

Algorithm:-

Binary search:-

- Fast search algorithm with time complexity of $O(\log n)$
- Based on divide and conquer.
- Data item should be sorted order.

Limitation

Example:-

10	14	19	26	31	42	44
0	1	2	3	4	5	6

Mid

V = 44

44 > 26

low = mid + 1
= 3 + 1 = 4

Mid = $\frac{4+6}{2} = 5$

low = mid - 1
= 6

Mid = $\frac{6+6}{2} = 6$

V = 14, A[mid] = 26

High = mid - 1 = 3 - 1 = 2

low = 0, Mid = $\frac{0+2}{2} = 1$

V = A[mid], A[1] = 14

location = 1

A[mid] =

A[6] = 44

while low ≤ High

do mid ← (low + high) / 2

If V = A[mid]

return mid

If V > A[mid]

low ← mid + 1

else high ← mid - 1

return NIL

Applications:-

- The searching techniques are applicable to a number of place in today's world, may be internet, search engines, on line enquiry, text pattern matching, finding a record from database etc.
- Most imp application of searching is to track a particular record from a large file, efficiently and

faster.

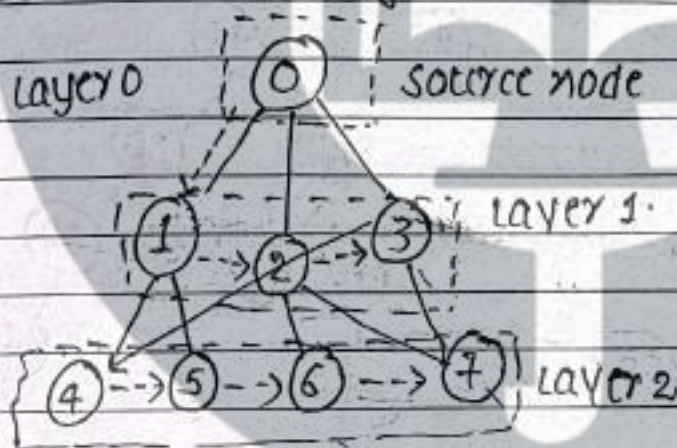
linear search
sequential search is made over all items one by one.

• Graphs Traversal:-

- Breadth First Search (BFS):- is a traversing algorithm where you should start traversing from a selected node and traverse graph layerwise thus exploring neighbour nodes (nodes which are directly connected to source node). You must then move towards next level neighbour nodes.

BFS suggests, you are required to traverse graph breadthwise.

- First move horizontally and visit all the nodes of the current layer.
- Move to the next layer.



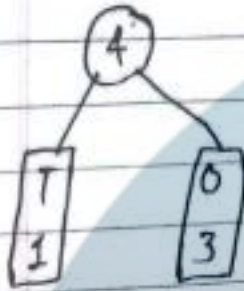
- Depth First Search (DFS):-

- Step 1: select a vertex in graph and make it source vertex and mark it visited.
- Step 2: find a vertex that is adjacent to source vertex and start a new search if it is not already visited.
- Step 3: Repeat step 2 using a new source vertex. When all

Min-Max Tree:

A	E	I	O	U	S	T
10	15	12	8	4	13	1

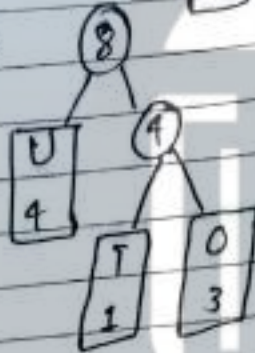
1.



A	E	I	U	S
10	15	12	4	13

A	E	I	S
10	15	12	13

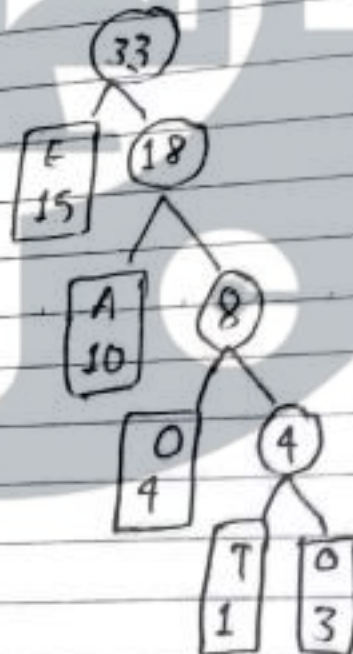
2.



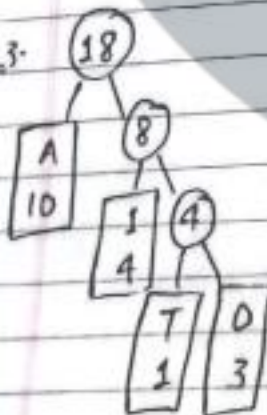
4.



5.

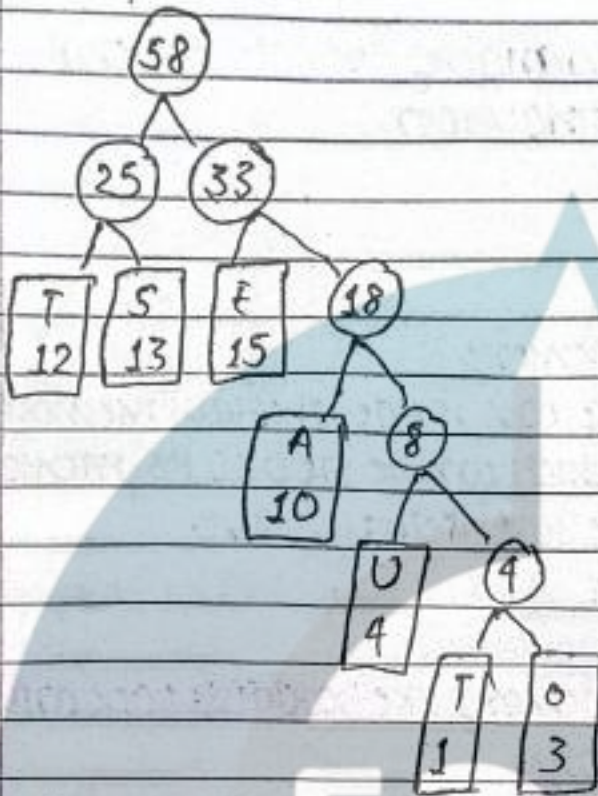


3.



E	T	S
15	12	13

6.



Binary search - certain ठाउँमा हुनुपर्छ ।
Final 10 → Required.

10	20	30	40	50	60	70
0	1	2	3	4	5	6

$L=0, H=6$

* $M = \frac{L+H}{2} = \frac{0+6}{2} = 3 \leftarrow 40$

* $L=0 \rightarrow$ Required value - 20
 $H = \text{Mid} - 1 = 3 - 1 = 2$

$M = \frac{L+H}{2} = \frac{0+2}{2} = 1 \leftarrow 20$

* $V=60 \rightarrow$ Required value - 60

$L=0$

$H=6$

$L = \text{Mid} + 1 = 4$

$L=0$
 $M = \frac{0+6}{2} = 3$
 $L = \text{Mid} + 1$
 $= 3 + 1 = 4$

$H=6$

$M = \frac{4+6}{2} = 5$
 $L=4$

$M = \text{Mid} - 1$
 $H=4$
 $L=4$

$M=4$

$M = \frac{4+6}{2} = 5$

Sorting (process of arranging the data either in ascending or descending order).

Types:-

- Internal sorting

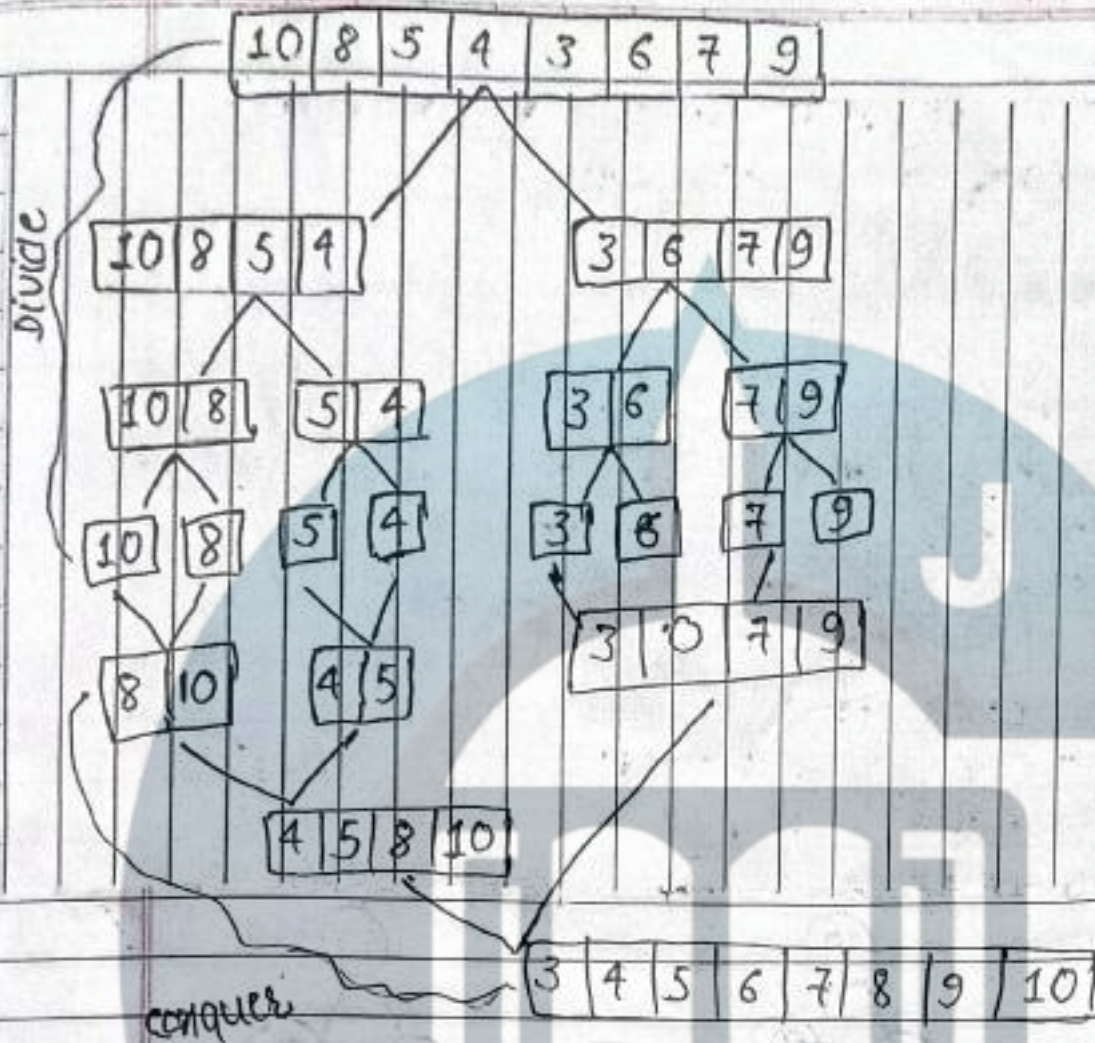
- is done in internal memory.
- The internal sorting can reside in main memory.
- Takes input only which can be fit into its memory.
- Eg:- Heap sort, Bubble sort, selection sort.

- External sorting

- is done in external memory like hard disks, tape drive.
- Use secondary memory.
- External sorting can take as much as large input.
- Merge sort.

✓ Merge sort:-

- is a sorting algorithm that follows divide and conquer approach. It works by recursively dividing the input array into smaller subarray and sorting those subarray then merging them back to obtained sorted array.



Merge Sort:

• Merge sort Algorithm:-

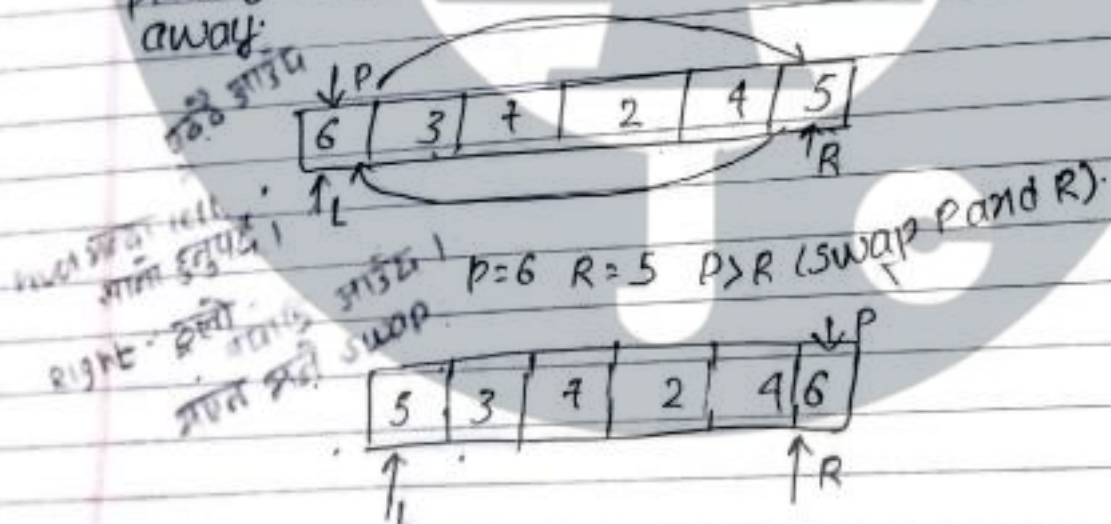
Step 1: If it is only one element in the list, consider it already sorted so needed.

Step 2: Divide the list recursively into two halves until it can no more be divided.

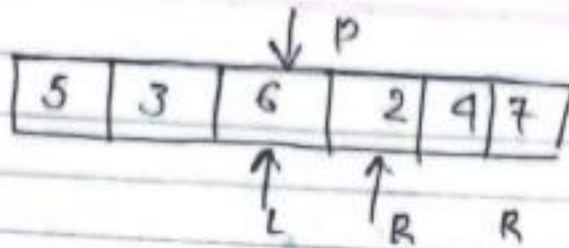
Step 3: Merge the smaller list into new list in sorted data.

• Quick sort:-

is a sorting algorithm based on the divide and conquer that picks an element as an pivot and partition the given array around the picked pivot by placing the pivot in its correct position in the sorted array.



$L = 5$ $L < P$ [MOVE L as step forward]
 $P = 6$



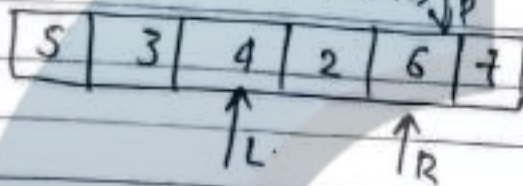
P = 6

R = 7

$P < R$ (Move pointers are step left).

P = 6

R = 4 (swap P and R)

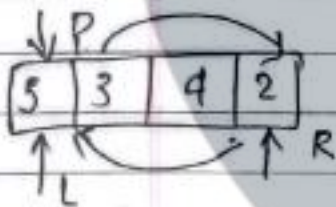
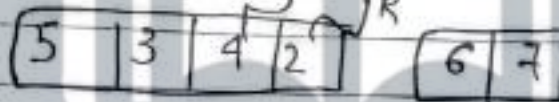
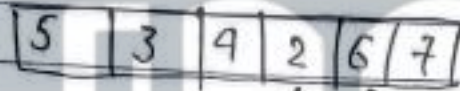


L = 4

P = 6

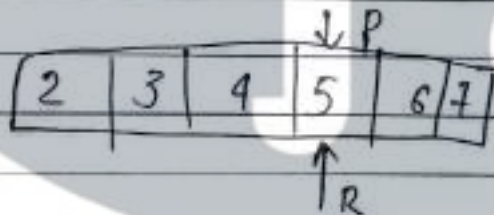
L = 7

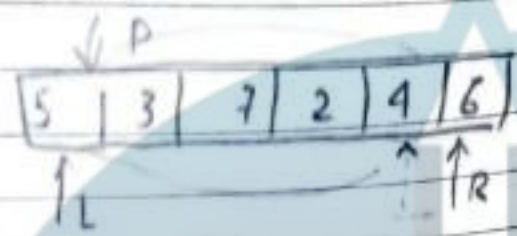
P = 6



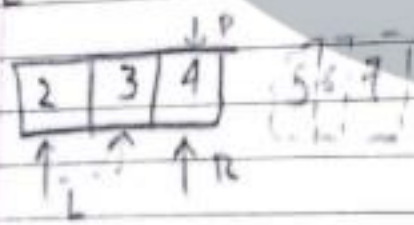
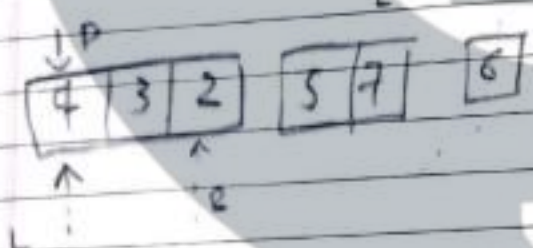
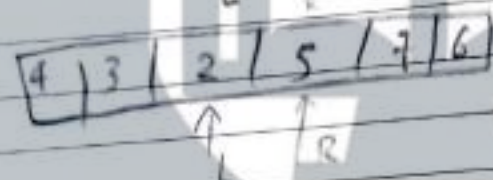
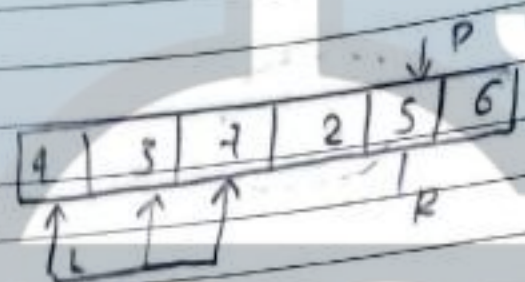
P = 5

R = 2





$P = 5$
 $R = 4$



Algorithm:-

- i) choose the highest index value as pivot.
- ii) Take two variables to point left and right of the list excluding pivot.
- iii) left ~~point~~^{point} to the low index.
- iv) Right point to the high.
- v) while value of left is less than pivot move right.
- vi) while value at right is greater than pivot move left.
- vii) If both step v and vi does not match left and right.
- viii) If left $\geq$ right, the point where they met is new pivot.

• Selection sort Algorithm:-

→ simple sorting algorithm. This sorting algorithm, ~~like insertion~~ works by repeatedly sorting elements. That is: we first find smallest value in array and exchange it with element in first position, then find second smallest element and exchange it with the element in second position, and we continue process in this way until the entire array is sorted.

Algorithm:-

- set MIN to location 0.
- search minimum element in list.
- swap with value at location MIN.
- increment MIN to point to next element.
- repeat until the list is sorted.

Radix sort:-

15, 1, 321, 10, 802, 2, 123, 90, 109, 11

→ 015, 001, 321, 010, 802, 002, 123, 090, 109, 0011

part 1

0 010, 090

1 001, 321, 011

2 802, 002

3 123

4

5 015,

6

7

8

9 109

part 2

0 001, 802, 002, 109

1 010, 011, 015

2 321, 123

3

4

5

6

7

8

9 090

part 3

0 001, 002, 010,

1 109, 123

2

3 321,

4

5

6

7

8 802

9

011, 015,
099

Now let's remove value we added:-

1, 2, 10, 11, 15, 90, 109, 123, 321, 802.

Note:- if elements do not have same no. of digits, find maximum no. of digits in an input element and add leading zeroes to elements having less digits. It does not change values of elements but still makes them k-digit numbers.

Radix sort:- is a stepwise sorting algorithm that starts the sorting from the least significant digit of the input elements. This sort assumes something about the input elements that they are all k-digit numbers.

The sorting starts with least significant digit of each element. These least significant digits are all considered individual elements and sorted first; followed by second least significant digits. This process is continued until all the digits of input elements are sorted.

Algorithm:-

Step 1:- check whether all the input elements have same number of digits. If not, check for numbers that have maximum no. of digits in list and add leading zeroes to ones that do not.

Step 2:- Take least significant digit of each element.

Step 3:- sort these digits using counting sort logic and change the order of elements based on output achieved.

Step 4:- Repeat the step 2 for next least significant digits until all the digits in elements are sorted.

Step 5:- final list of elements achieved after kth loop is the sorted output.

✓ Bubble sort: is a simple sorting algorithm. It is a comparison based algorithm in which each pair of adjacent elements is compared and elements are swapped if they are not in order. This algorithm is not suitable for large data sets as its average and worst case complexity are of $O(n^2)$ where n is the number of items.

Algorithm:-

- Step 1:- check if 1st element in input array is greater than the next element in array.
- Step 2:- If it is greater, swap 2 elements; otherwise move the pointer forward in array.
- Step 3:- Repeat step 2 until we reach end of the array.
- Step 4:- check if the elements are sorted; if not repeat same process (step 1 to step 3) from last element of the array to the first.
- Step 5:- Final output achieved is the sorted array.

Pass 1

7	2	12	8	3
0	1	2	3	4

Diagram showing the first pass of bubble sort. The array is [7, 2, 12, 8, 3] with indices 0 to 4. An arrow points from index 0 to index 1, indicating a swap.

2	7	12	8	3
0	1	2	3	4

Diagram showing the array after the first swap: [2, 7, 12, 8, 3]. An arrow points from index 2 to index 3, indicating a swap.

2	7	8	12	3
0	1	2	3	4

Diagram showing the array after the second swap: [2, 7, 8, 12, 3].

Pass 2

2	7	8	3	12
0	1	2	3	4

Diagram showing the array after the first swap of the second pass: [2, 7, 8, 3, 12]. The element 12 is circled, and an arrow points from index 3 to index 4, indicating a swap.

2	7	3	8	12
---	---	---	---	----

Diagram showing the array after the second swap of the second pass: [2, 7, 3, 8, 12].

Pass 3

2	3	7	8	12
---	---	---	---	----

Diagram showing the final sorted array: [2, 3, 7, 8, 12].

∴ Bubble sort

Shell sort:- highly efficient sorting algorithm and is based on insertion sort algorithm. This algorithm avoids large shifts as in case of insertion sort, if smaller value to the far right and has to be moved to the far left.

This algorithm uses insertion sort on widely spread elements first to sort them and then sorts the less widely spaced elements. This spacing is termed as interval. This interval is calculated based on Knuth's formula as:-

$$h = h * 3 + 1$$

where

h is interval with initial value 1.

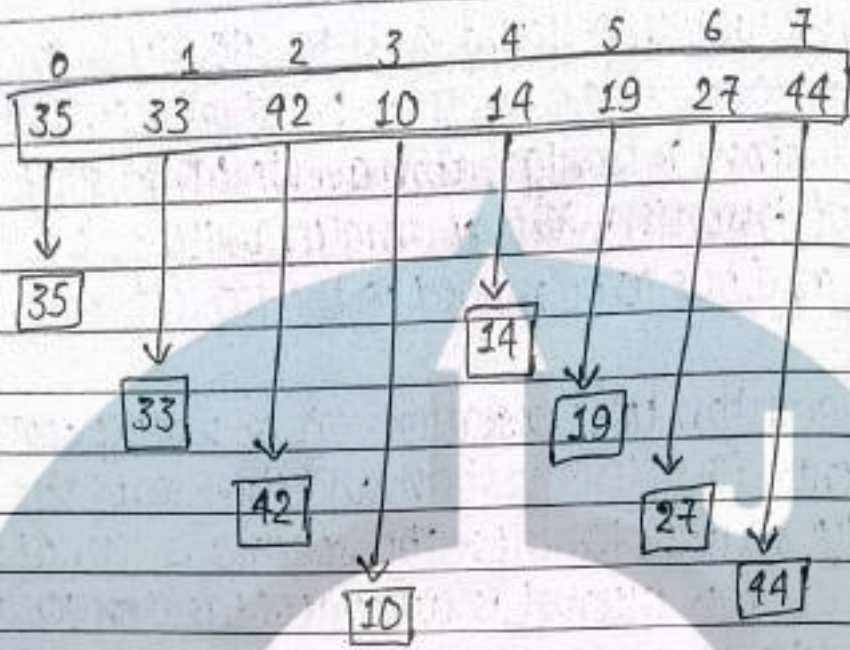
Algorithm:-

1. Initialize value of h .
2. Divide list into smaller sub-list of equal interval h .
3. Sort these sub-lists using insertion sort.
4. Repeat until complete list is sorted.

00107010012699

Example:-

(interval of 4).

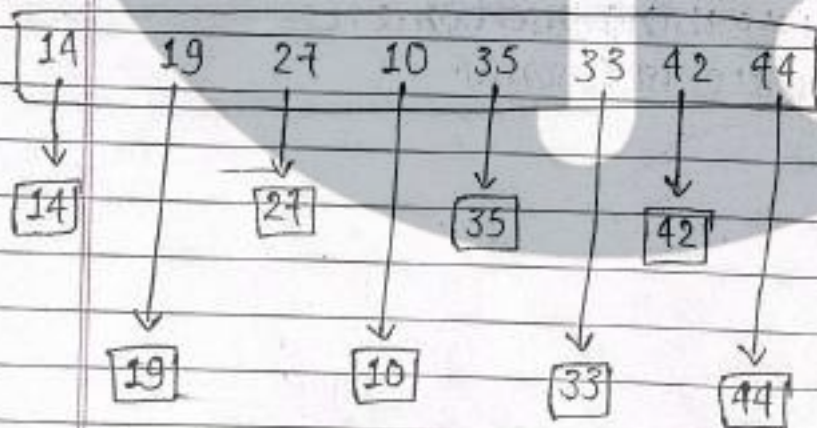


We compare values in each sub list and swap them (if necessary) in original array.

Now, new array:-

0	1	2	3	4	5	6	7
14	19	27	10	35	33	42	44

Then, we take interval of 2 and this gap generates 2 sub-lists
[14, 27, 35, 42], [19, 10, 33, 44]



Compare and swap values, if required in original array.
After this step, array should look like this:-

0	1	2	3	4	5	6	7
14	19	27	10	35	33	42	44

We sorted rest of array using interval of 1. shell sort uses insertion sort to sort array.

chapter-11 (Algorithms)

1. Divide and conquer Algorithms.
2. Deterministic and Non-Deterministic.
3. Serial vs Parallel Algorithm.
4. Heuristic and Approximate Algorithms.

$I = 00$
 $S = 01$
 $E = 10$
 $U = 1100$
 $A = 111$
 $T = 11010$
 $O = 11011$

Sorting Algorithms	Time complexity			space complexity
	Best case	Average case	Worst case	Worst case
Bubble sort	$\Omega(N)$	$O(N^2)$	$O(N^2)$	$O(1)$
Selection sort	$\Omega(N^2)$	$O(N^2)$	$O(N^2)$	$O(1)$
Insertion sort	$\Omega(N)$	$O(N^2)$	$O(N^2)$	$O(1)$
Quick sort	$\Omega(N \log N)$	$O(N \log N)$	$O(N^2)$	$O(N)$
Merge sort	$\Omega(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(N)$
Heap sort	$\Omega(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(1)$

Heap sort - is an efficient sorting technique based on the heap data structure. The heap is nearly complete binary tree where the parent node could either be minimum or maximum. The heap with minimum root node is called min heap and the root node with maximum root node is max heap.

• Algorithm:-

Algorithm: Heapsort (A)

BUILD-MAX-HEAP(A)

for $i = A.length$ down to 2

exchange $A[1]$ with $A[i]$

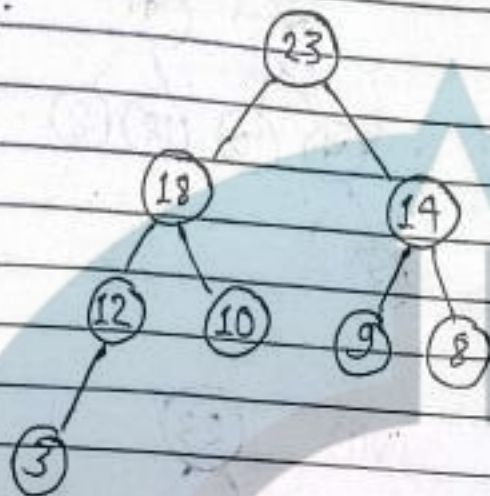
$A.heap-size = A.heap-size - 1$

MAX-HEAPIFY(A, 1)

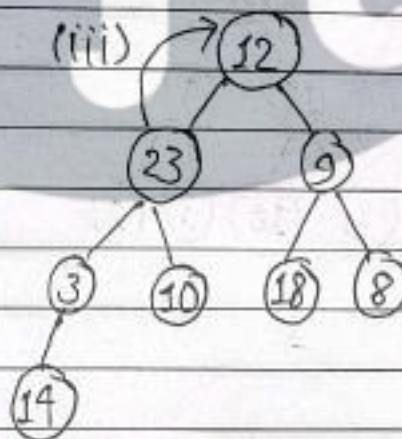
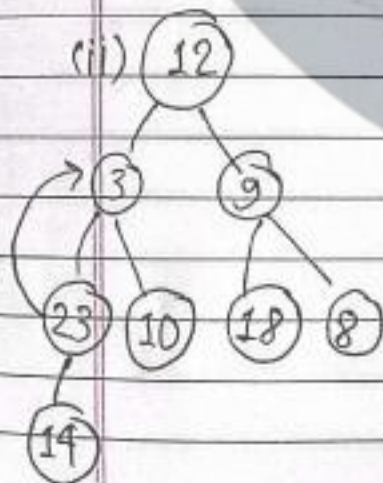
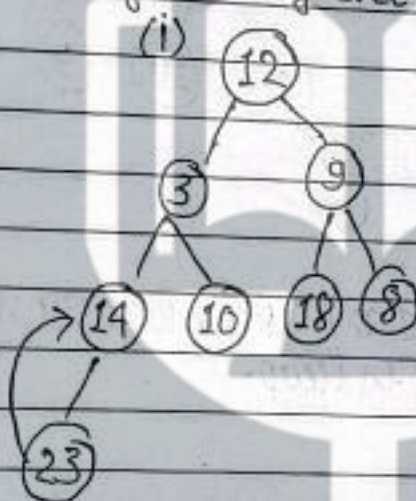
(Sorting algorithm first constructs heap A[1] by calling Build-Max-Heap algorithm & removes root element to swap it with minimum valued node at leaf. Then this method is applied to rearrange elements accordingly).

12	3	9	14	10	18	8	23
----	---	---	----	----	----	---	----

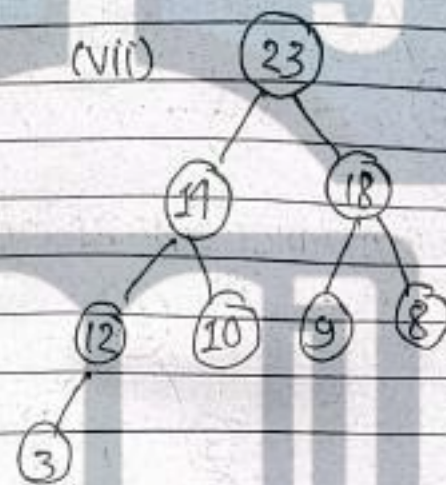
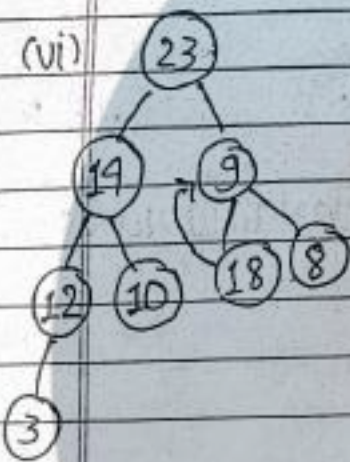
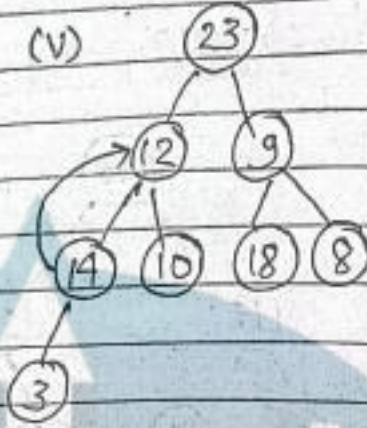
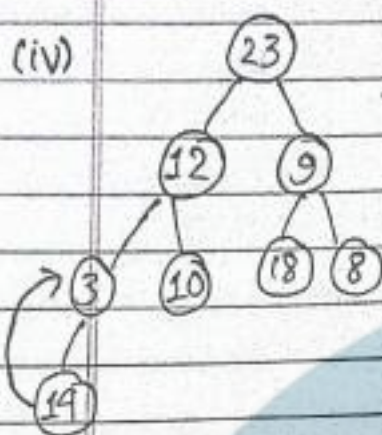
step 1:-



step 2: Rearrange binary tree by exchanging nodes.

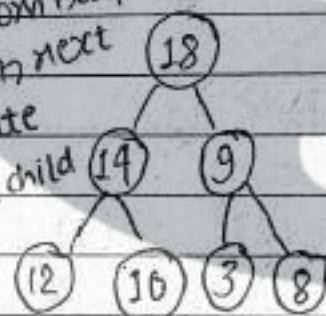


20/11/20



Root Node is 23, so 23 is popped and 18 is made the next root because it is next maximum node in Heap.

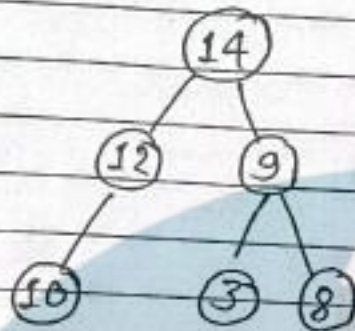
Remove root node from heap and replace it with next immediate maximum value child of root.



(23)

23

18 is popped after 23 which is replaced by 14.



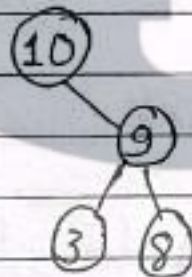
(18) (23)

current root 14 is popped from heap and is replaced by 12.



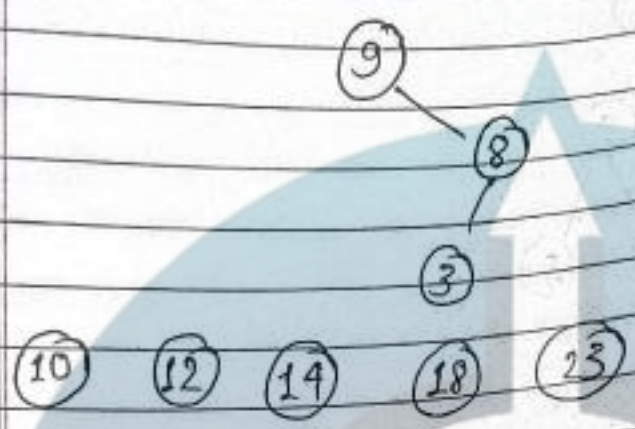
(14) (18) (23)

12 is popped and replaced with 10.



(12) (14) (18) (23)

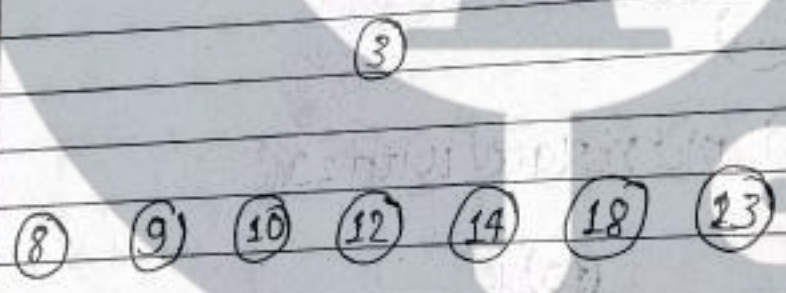
Similarly, all other elements are popped using same process.



Here, current root element 9 is popped and elements 8 and 3 are remained in the tree.



Then, 8 will be popped leaving 3 in tree.



Now, sorted elements:-

3 8 9 10 12 14 18 23.